

INNOPOLIS UNIVERSITY

Введение в компьютерное зрение

Илья Афанасьев, к.т.н., ЛИРС

Лекция 7 · Границы, линии и робастное оценивание

Dr. Ilya Afanasyev, i.afanasyev@innopolis.ru

«Introduction to Computer Vision» course, Innopolis University (IU), Russia, Fall 2026

Карта курса

Блок I Изображение и его источник	Блок II Операции над изображением	Блок III Признаки	Блок IV Гомография и геометрия	Блок V Видео и движение
L1 Введение. Изображение и видео как данные	**L4** Геометрические преобразования	**L7** Границы, линии и робастное оценивание	**L10** Гомография: оценивание и выравнивание	**L13** Видео: движение и оптический поток
L2 Формирование изображения: оптика и сенсор	**L5** Пространственная фильтрация	**L8** Точки интереса: детекторы	**L11** Модель камеры и калибровка	**L14** Трекинг и распознавание объектов
L3 Цвет, гистограммы и точечные операции	**L6** Частотная область и пирамиды	**L9** Дескрипторы и сопоставление	**L12** Стерео и эпиполярная геометрия	**L15** Синтез: от двух видов к 3D. Итоги

Сегодня — L7: Границы, линии и робастное оценивание

Опоры из L5–L6 и разбор selfcheck

RECAP

- Свёртка и спектр ядра: box «звенит» sinc-лепестками, Гаусс — нет; свёртка = умножение спектров
- Производная — это тоже свёртка: ядро [1 0 −1]/2; теорема о производной свёртки из L6: d/dx(G*I) = (dG/dx)*I
- Unsharp усилил шум в 1.94 раза (L6): дифференцирование и шум — враги; сегодня посчитаем цену точно
- Разбор selfcheck L6: поворот спектра; алиасинг 5 px→5 px и σ 1.2; АЧХ [1,2,1]/4 = (1+cos2πf)/2 (ФНЧ); склейка при перекрытии 200 — ≤5 уровней

Сегодня производная перестаёт быть абстракцией: из неё соберём край, прямую и модель

Чему научимся сегодня

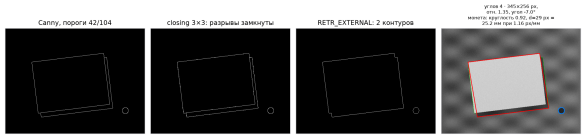
LEARNING OBJECTIVES

- Посчитать градиент и его направление: Sobel, Scharr, `magnitude/phase`; объяснить, почему Собель сглаживает поперёк
- Обосновать сглаживание числом: SNR пика края до и после Гаусса (3.3 → 5.0 → 7.8 при σ 10 шума)
- Собрать Canny по шагам (NMS + гистерезис) и выбрать пороги по гистограмме градиента, а не «из туториала»
- Найти прямые преобразованием Хафа: построить аккумулятор вручную, настроить `HoughLinesP`, схлопнуть дубли
- Оценить итерации RANSAC по формуле $N = \ln(1-p)/\ln(1-\varepsilon^s)$ и сравнить МНК, Huber и RANSAC на выбросах

Найти лист бумаги на столе

МОТИВАЦИЯ

- Дано: фото листа на clutter-фоне (`doc-table.jpg`), плюс монета рядом. Найти: четыре угла листа и размеры в мм
- Наивно: порог яркости — F1 0.41 (тени, блики); порог $|\nabla I|$ одним числом — 0.62; Canny с динамическими порогами + контуры — 0.93
- Прямые края листа: 17 отрезков Хафа → 4 стороны после кластеризации; углы — пересечения; контроль: RANSAC отбрасывает блики
- Это и есть ДЗ 2 «Сканер документов»: края → линии → геометрия, 12 мс на кадр



© синтетика «лист на столе» и её контур — собственная иллюстрация

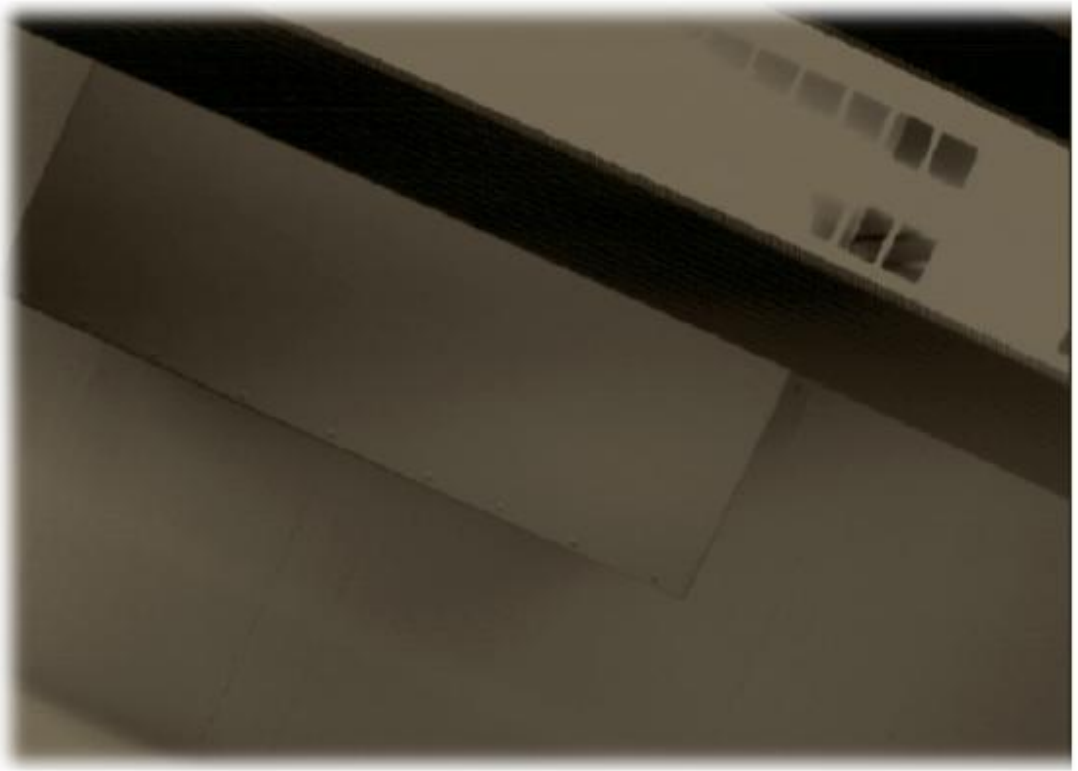
1

Край: градиент и Canny

Профили краёв · производная и шум · маски Собеля · вторая производная · NMS · гистерезис · пороги по гистограмме · контуры

Edge detection

- Ultimate goal of edge detection: an idealized line drawing.
- Edge contours in the image correspond to important scene contours.



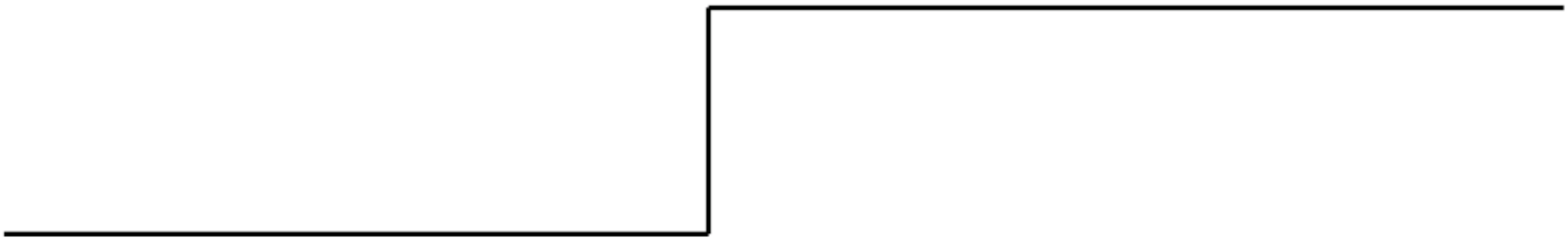


- Края на изображении (**Edges**) — это места, где наблюдаются резкие изменения яркости или цвета. Эти изменения могут указывать на границы между разными объектами или поверхностями.
- Обнаружение краев уменьшает количество информации на изображении, так как фокусируется только на значительных изменениях, игнорируя однородные области. Это помогает упростить анализ изображения.

© M. Chli, P. Furgale, M. Hutter, M. Rufli, D. Scaramuzza, R. Siegwart, Autonomous Mobile Robots, ETH, 2016, http://www.asl.ethz.ch/education/lectures/autonomous_mobile_robots.html

Edge detection: edge = intensity discontinuity in 1 direction

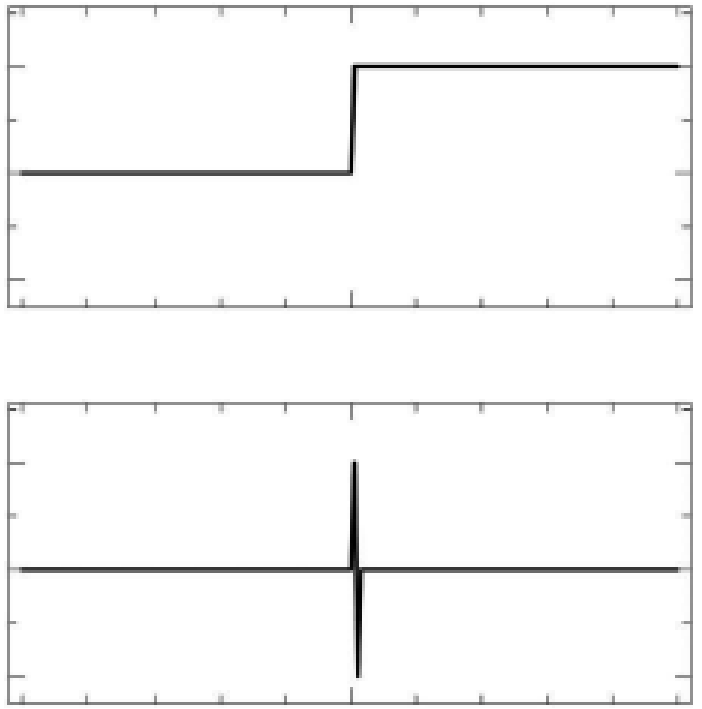
- Edges correspond to sharp changes of intensity
- How to detect an edge?
  - Change is measured by 1st order derivative in 1D
  - Big intensity change ⇨ magnitude of derivative is large
  - Or 2nd order derivative is zero.



© M. Chli, P. Furgale, M. Hutter, M. Rufli, D. Scaramuzza, R. Siegwart, Autonomous Mobile Robots, ETH, 2016, http://www.asl.ethz.ch/education/lectures/autonomous_mobile_robots.html

Edge detection: edge = intensity discontinuity in 1 direction

- Edges correspond to sharp changes of intensity
- How to detect an edge?
  - Change is measured by 1st order derivative in 1D
  - Big intensity change ⇨ magnitude of derivative is large
  - Or 2nd order derivative is zero.

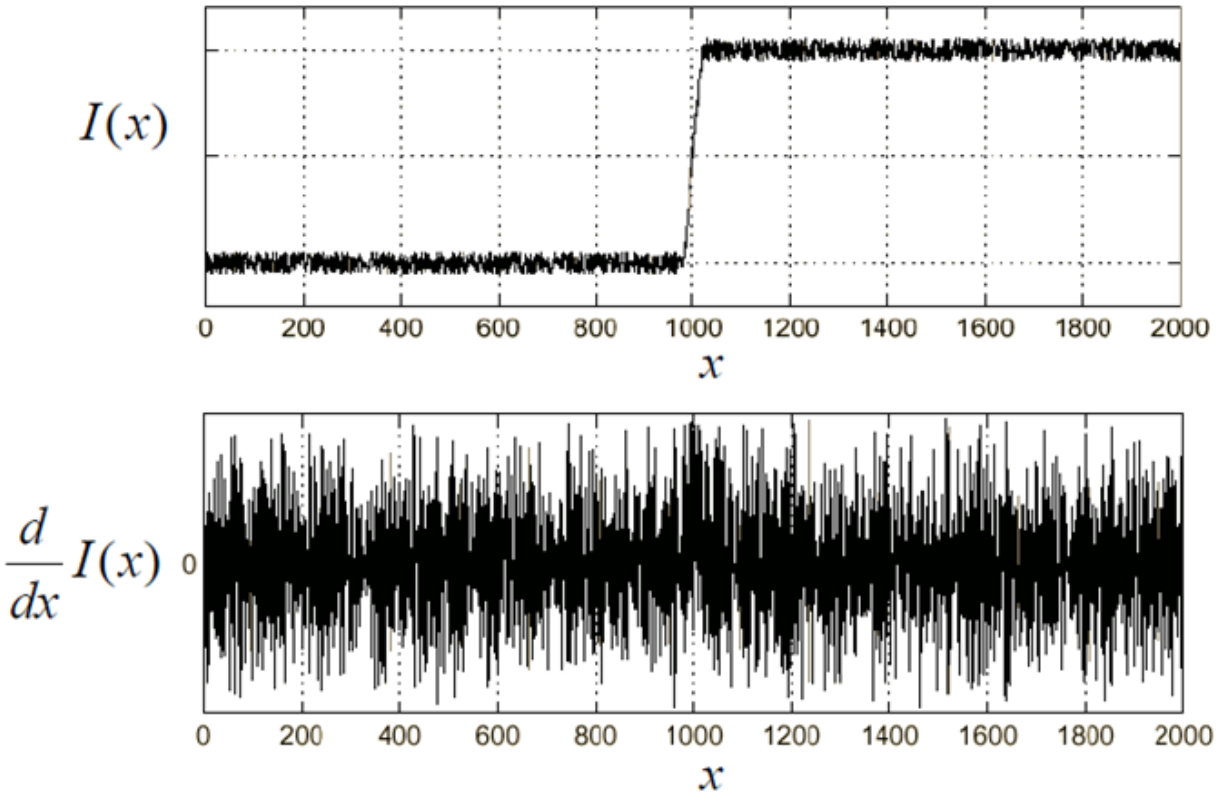


- **Простой детектор краев** (**Naive Edge Detector**) основан на дифференцировании, то есть вычисления производной интенсивности пикселей изображения.
- **Дифференцирование** (**вычисление градиента яркости**) показывает, насколько сильно и в каком направлении изменяется интенсивность в каждой точке изображения.
- **Переходы в интенсивности** (Transitions in Intensity) могут соответствовать границам объектов или контуров на изображении.

© M. Chli, P. Furgale, M. Hutter, M. Rufli, D. Scaramuzza, R. Siegwart, Autonomous Mobile Robots, ETH, 2016, http://www.asl.ethz.ch/education/lectures/autonomous_mobile_robots.html

1D edge detection

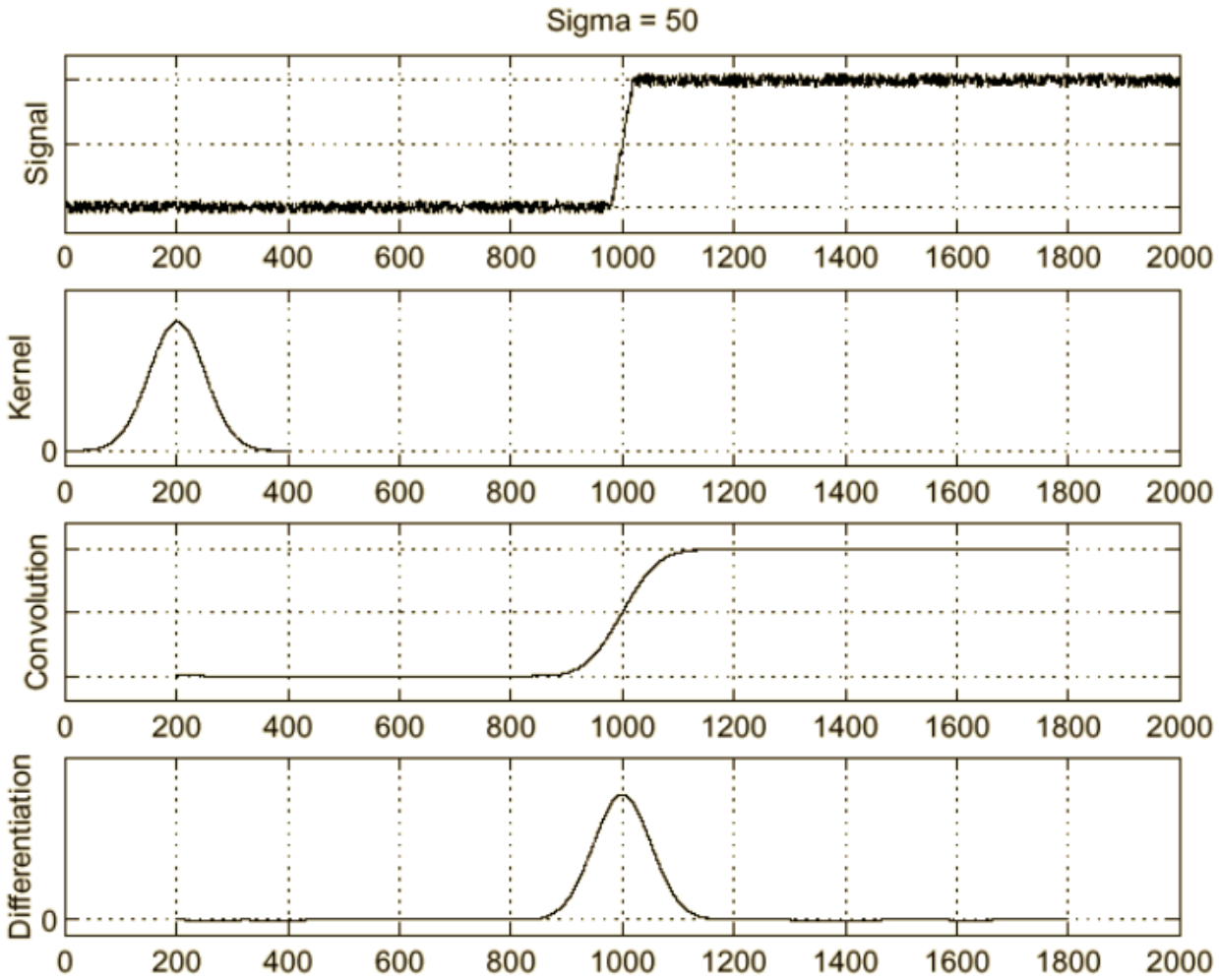
- Consider a single row or column of the image, where image intensity shows an obvious change



- **Дифференцирование зашумленного сигнала** камеры приводит к появлению дополнительных пиков, которые могут сильно усложнить обнаружение краев.
- Гораздо более стабильный сигнал производной можно получить, предварительно обработав сигнал камеры с помощью **гауссовской функции сглаживания**.

© M. Chli, P. Furgale, M. Hutter, M. Rufli, D. Scaramuzza, R. Siegwart, Autonomous Mobile Robots, ETH, 2016, http://www.asl.ethz.ch/education/lectures/autonomous_mobile_robots.html

1D edge detection: solution - smooth first



$I(x)$

$g_\sigma(x)$

$s(x) = I(x) * g_\sigma(x)$

$s'(x) = \frac{d}{dx}\big(s(x)\big)$

- Where is the edge?

Edges occur at maxima/minima of $s'(x)$

© M. Chli, P. Furgale, M. Hutter, M. Rufli, D. Scaramuzza, R. Siegwart, Autonomous Mobile Robots, ETH, 2016, http://www.asl.ethz.ch/education/lectures/autonomous_mobile_robots.html

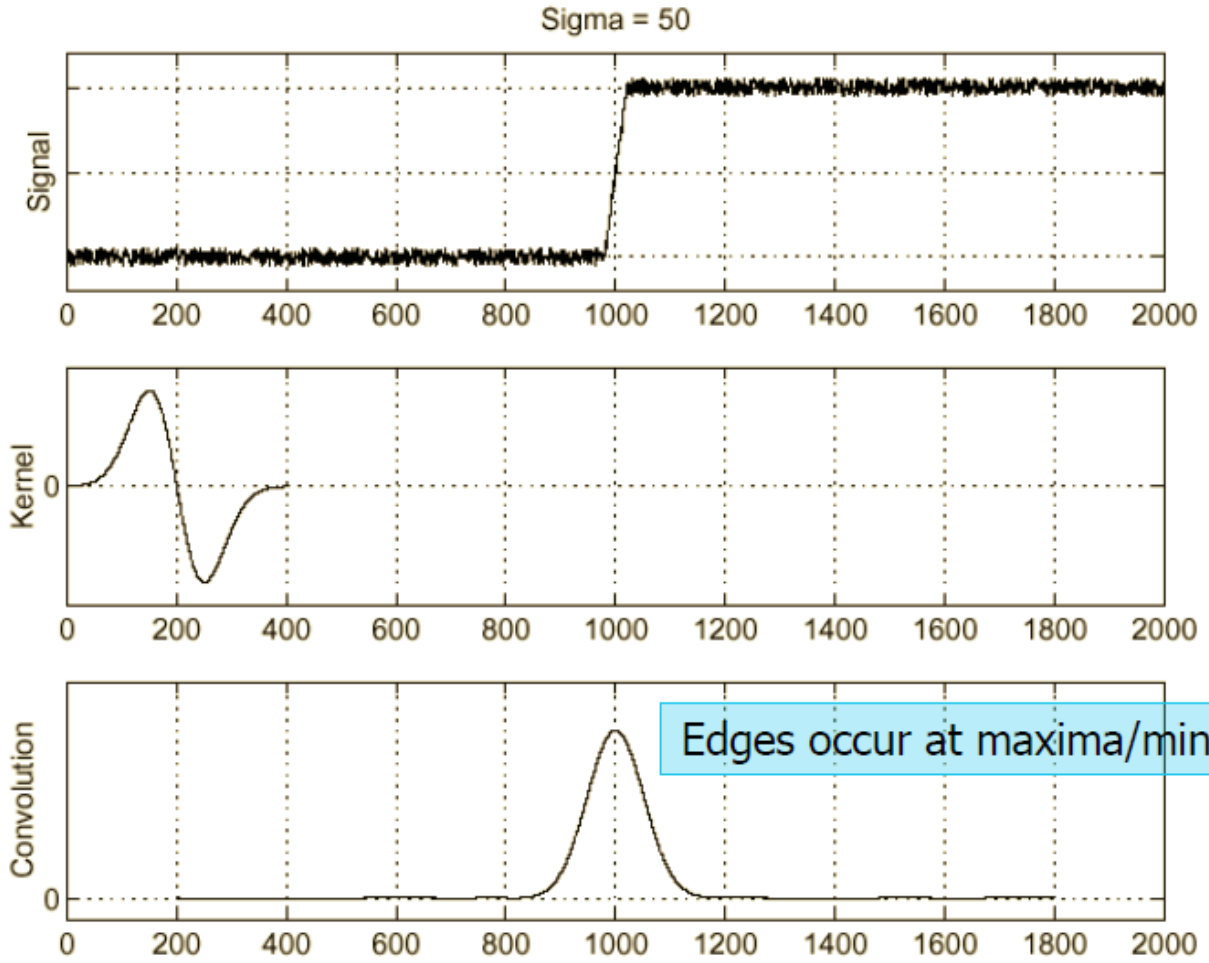
1D edge detection: derivative theorem of convolution

- $$s'(x) = \frac{d}{dx}\left(g_\sigma(x) * I(x)\right) = g'_\sigma(x) * I(x)$$
- This saves us one operation:

$$I(x)$$

$$g'_\sigma(x) = \frac{d}{dx} g_\sigma(x)$$

$$s'(x) = g'_\sigma(x) * I(x)$$



© M. Chli, P. Furgale, M. Hutter, M. Rufli, D. Scaramuzza, R. Siegwart, Autonomous Mobile Robots, ETH, 2016, http://www.asl.ethz.ch/education/lectures/autonomous_mobile_robots.html

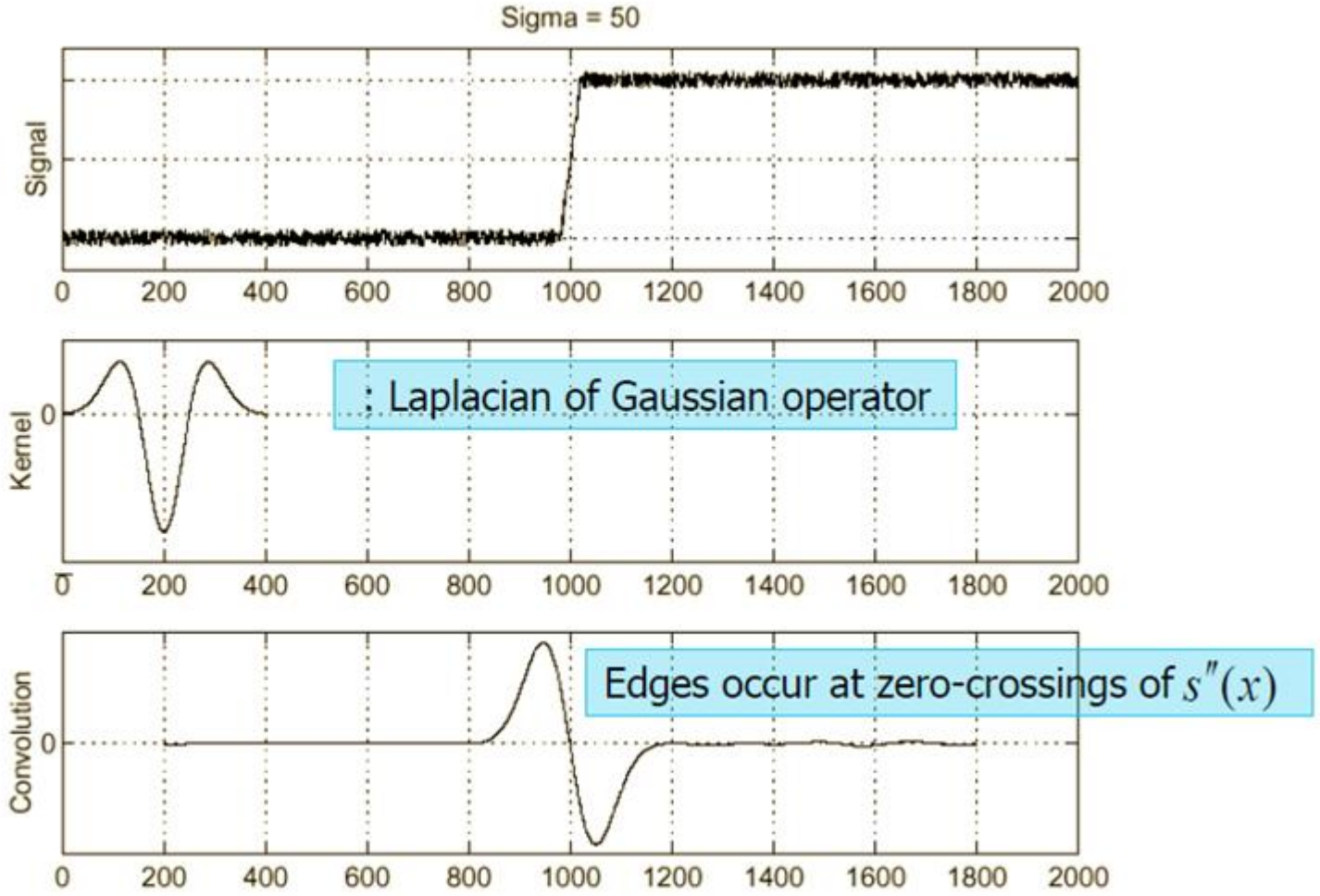
1D edge detection: zero-crossings

- Locations of Maxima/minima in $s'(x)$ are equivalent to zero-crossings in $s''(x)$

$$I(x)$$

$$g''_\sigma(x) = \frac{d^2}{dx^2} g_\sigma(x)$$

$$s''(x) = g''_\sigma(x) * I(x)$$



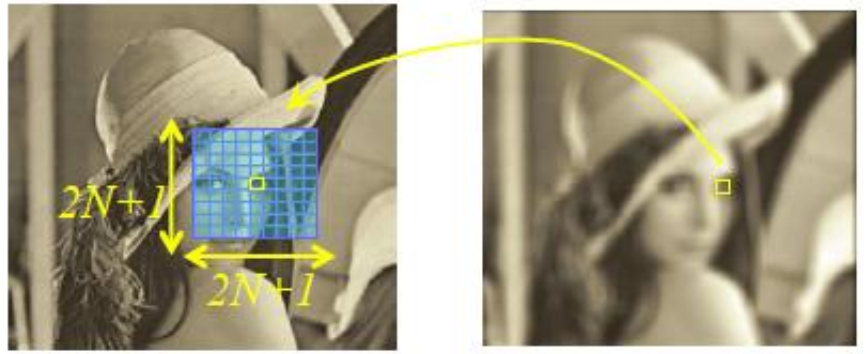
© M. Chli, P. Furgale, M. Hutter, M. Rufli, D. Scaramuzza, R. Siegwart, Autonomous Mobile Robots, ETH, 2016, http://www.asl.ethz.ch/education/lectures/autonomous_mobile_robots.html

Keypoints on smoothing + derivative masks

Smoothing masks

$$\begin{bmatrix} \frac{1}{9} & \frac{1}{9} & \frac{1}{9} \\ \frac{1}{9} & \frac{1}{9} & \frac{1}{9} \\ \frac{1}{9} & \frac{1}{9} & \frac{1}{9} \end{bmatrix}$$

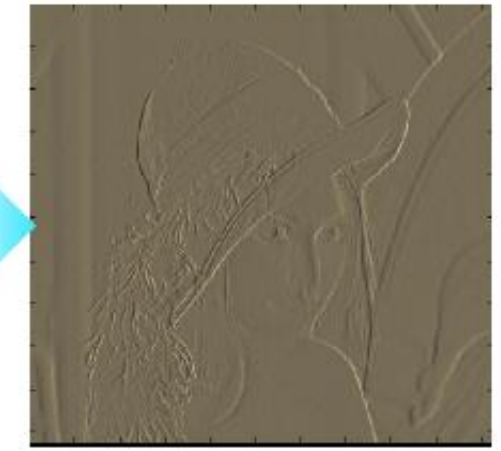
- Values positive
- Always **sum to 1** → constant regions same as input
- Amount of **smoothing proportional to mask size**



Derivative masks

- Opposite signs used to get high response in regions of high contrast
- Always **sum to 0** → no response in constant regions
- High absolute value at points of high contrast

$$F_x = \begin{pmatrix} 1 & -1 \end{pmatrix}$$

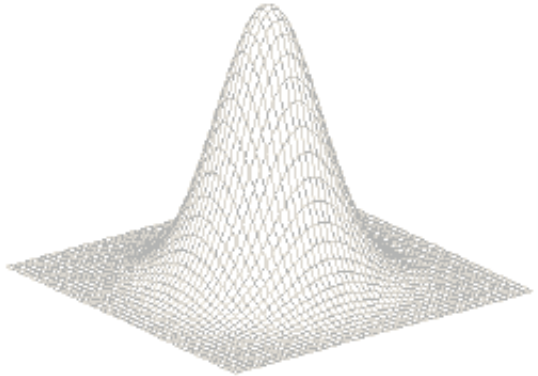


Sobel Filter

$$F_x = \begin{array}{|c|c|c|} \hline -1 & 0 & 1 \\ \hline -2 & 0 & 2 \\ \hline -1 & 0 & 1 \\ \hline \end{array}$$

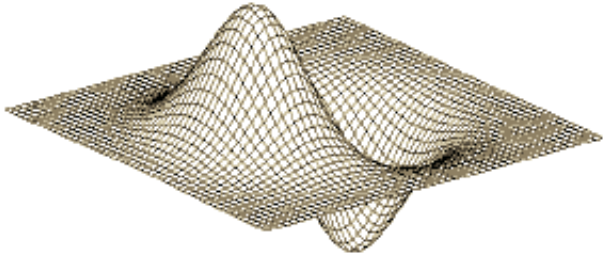
© M. Chli, P. Furgale, M. Hutter, M. Rufli, D. Scaramuzza, R. Siegwart, Autonomous Mobile Robots, ETH, 2016, http://www.asl.ethz.ch/education/lectures/autonomous_mobile_robots.html

2D edge detection: other approx. of derivative filters



Gaussian

$$G_\sigma(u,v) = \frac{1}{2\pi\sigma^2} e^{\frac{u^2+v^2}{\sigma^2}}$$



derivative of Gaussian

$$\nabla G_\sigma(u,v) = \begin{pmatrix} \frac{\partial}{\partial u} G_\sigma(u,v) \\ \frac{\partial}{\partial v} G_\sigma(u,v) \end{pmatrix}$$

Laplacian of Gaussian



$$\nabla^2 G_\sigma(u,v) = \begin{pmatrix} \frac{\partial^2}{\partial u^2} G_\sigma(u,v) \\ \frac{\partial^2}{\partial v^2} G_\sigma(u,v) \end{pmatrix}$$

© M. Chli, P. Furgale, M. Hutter, M. Rufli, D. Scaramuzza, R. Siegwart, Autonomous Mobile Robots, ETH, 2016, http://www.asl.ethz.ch/education/lectures/autonomous_mobile_robots.html

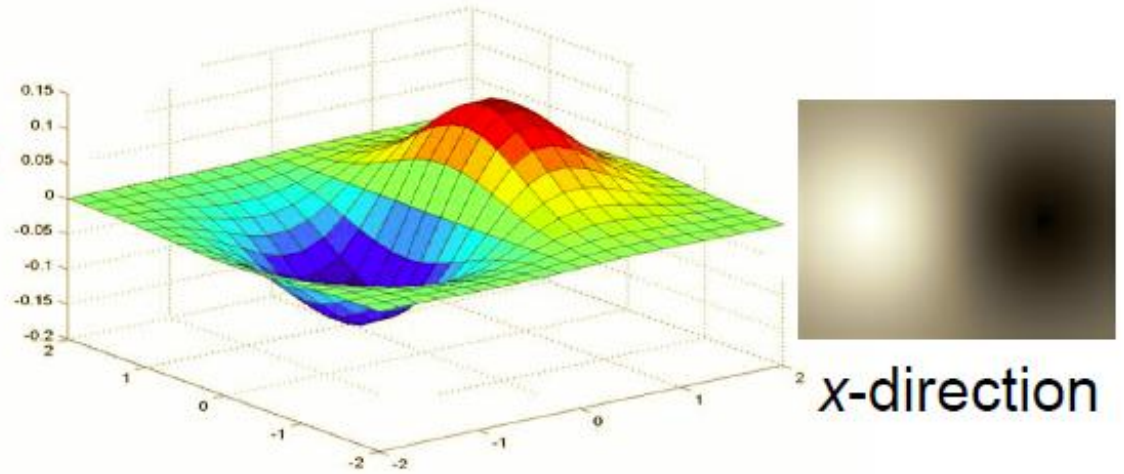
2D edge detection: other approx. of derivative filters

$$F_x = \begin{pmatrix} 1 & -1 \end{pmatrix}$$

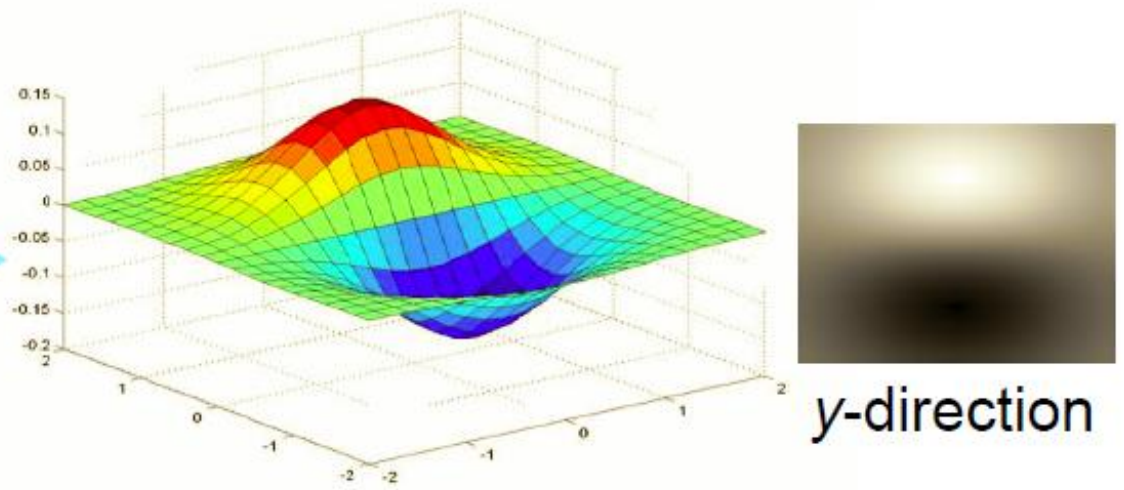
$$\nabla S = \nabla * G * I = \begin{pmatrix} (F_x * G) * I \\ (F_y * G) * I \end{pmatrix}$$

$$\begin{pmatrix} 0.0030 & 0.0133 & 0.0219 & 0.0133 & 0.0030 \\ 0.0133 & 0.0596 & 0.0983 & 0.0596 & 0.0133 \\ 0.0219 & 0.0983 & 0.1621 & 0.0983 & 0.0219 \\ 0.0133 & 0.0596 & 0.0983 & 0.0596 & 0.0133 \\ 0.0030 & 0.0133 & 0.0219 & 0.0133 & 0.0030 \end{pmatrix}$$

$$F_y = \begin{pmatrix} 1 \\ -1 \end{pmatrix}$$



x-direction



y-direction

© M. Chli, P. Furgale, M. Hutter, M. Rufli, D. Scaramuzza, R. Siegwart, Autonomous Mobile Robots, ETH, 2016, http://www.asl.ethz.ch/education/lectures/autonomous_mobile_robots.html

2D edge detection

- Find gradient of smoothed image in both directions

Usually use a separable filter such that:
$G_\sigma(x,y) = g_\sigma(x) g_\sigma(y)$

$$\nabla S = \nabla (G_\sigma * I) = \begin{bmatrix} \frac{\partial (G_\sigma * I)}{\partial x} \\ \frac{\partial (G_\sigma * I)}{\partial y} \end{bmatrix} = \begin{bmatrix} \frac{\partial G_\sigma}{\partial x} * I \\ \frac{\partial G_\sigma}{\partial y} * I \end{bmatrix} = \begin{bmatrix} g'_\sigma(x) g_\sigma(y) * I \\ g_\sigma(x) g'_\sigma(y) * I \end{bmatrix}$$

- Discard pixels with $|\nabla S|$ (i.e. edge strength) below a certain threshold
- **Non-maxima suppression**: identify local maxima of $|\nabla S|$
  ⇨ detected edges

© M. Chli, P. Furgale, M. Hutter, M. Rufli, D. Scaramuzza, R. Siegwart, Autonomous Mobile Robots, ETH, 2016, http://www.asl.ethz.ch/education/lectures/autonomous_mobile_robots.html

2D edge detection by Canny detector

- **Детектор краев Кэнни** (Canny edge detector)
  - это один из наиболее эффективных и широко используемых алгоритмов обнаружения краев в компьютерном зрении.
  - был разработан Джоном Кэнни в 1983 году и до сих пор остается эталоном для других методов обнаружения краев.
- Три цели детектора Кэнни:
  - Максимизация отношения сигнал/шум;
  - Высокая точность в определении местоположения краев;
  - Минимизация количества ложных ответов для каждого реального края.
- Детектор Кэнни начинает с **гауссовского сглаживания изображения**, чтобы уменьшить шум.
- После сглаживания детектор ищет **максимумы в производной** (**градиенте**) интенсивности изображения, детектируя края.
- На практике часто **сглаживание** и **дифференцирование объединяются** в одну операцию для повышения эффективности, что упрощает процесс и ускоряет выполнение алгоритма

$$(G * I)' = G' * I$$

Таким образом, сглаживание изображения с помощью свертки с гауссовой функцией, а затем дифференцирование, эквивалентно свертке изображения с первой производной Гауссовой функции.

2D edge detection: partial derivatives of an image



$$S_x = \frac{\partial (G_\sigma * I)}{\partial x}$$

$$F_x = \begin{array}{|c|c|}\hline -1 & 1 \\ \hline \end{array}$$



$$S_y = \frac{\partial (G_\sigma * I)}{\partial y}$$

$$F_y = \begin{array}{|c|}\hline -1 \\ \hline 1 \\ \hline \end{array}$$



$$|\nabla S| = \sqrt{S_x^2 + S_y^2}$$



thresholding

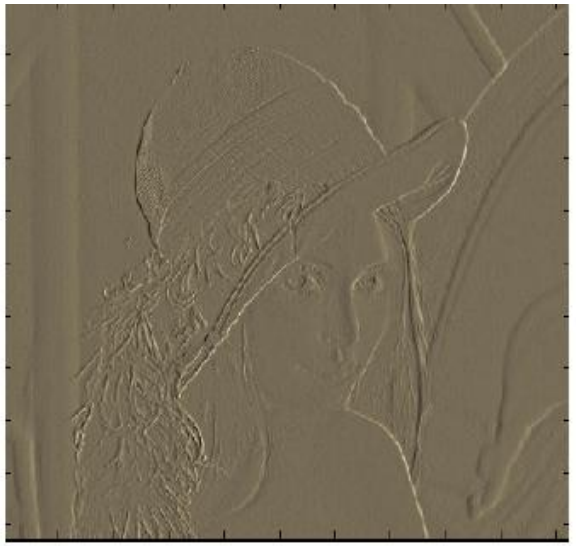
© M. Chli, P. Furgale, M. Hutter, M. Rufli, D. Scaramuzza, R. Siegwart, Autonomous Mobile Robots, ETH, 2016, http://www.asl.ethz.ch/education/lectures/autonomous_mobile_robots.html

2D edge detection: example with Canny edge detector

$$|\nabla S| = \sqrt{S_x^2 + S_y^2}$$



$|\nabla S|$: Edge strength



$$S_x = \frac{\partial (G_\sigma * I)}{\partial x}$$

$$\nabla S = \nabla \left(G_\sigma * I \right)$$



$$S_y = \frac{\partial (G_\sigma * I)}{\partial y}$$

© M. Chli, P. Furgale, M. Hutter, M. Rufli, D. Scaramuzza, R. Siegwart, Autonomous Mobile Robots, ETH, 2016, http://www.asl.ethz.ch/education/lectures/autonomous_mobile_robots.html

2D edge detection: example with Canny edge detector



Thresholding $|\nabla S|$

© M. Chli, P. Furgale, M. Hutter, M. Rufli, D. Scaramuzza, R. Siegwart, Autonomous Mobile Robots, ETH, 2016, http://www.asl.ethz.ch/education/lectures/autonomous_mobile_robots.html

2D edge detection: example with Canny edge detector

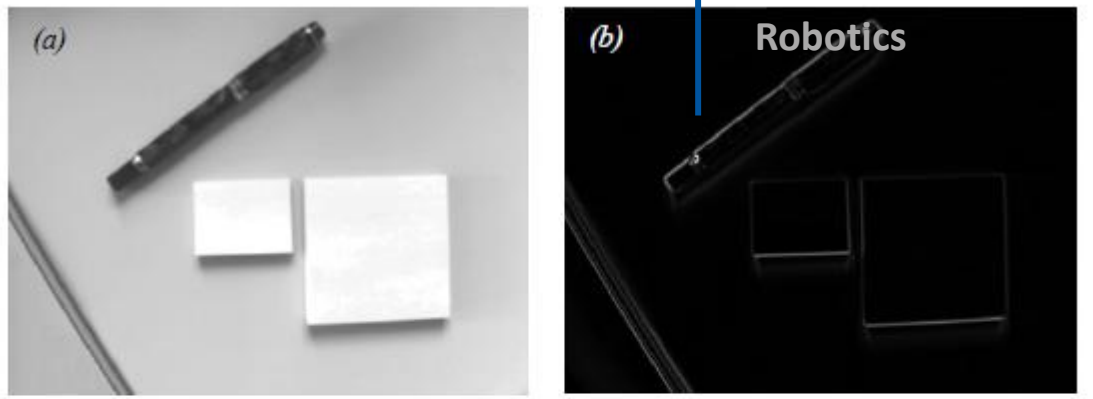


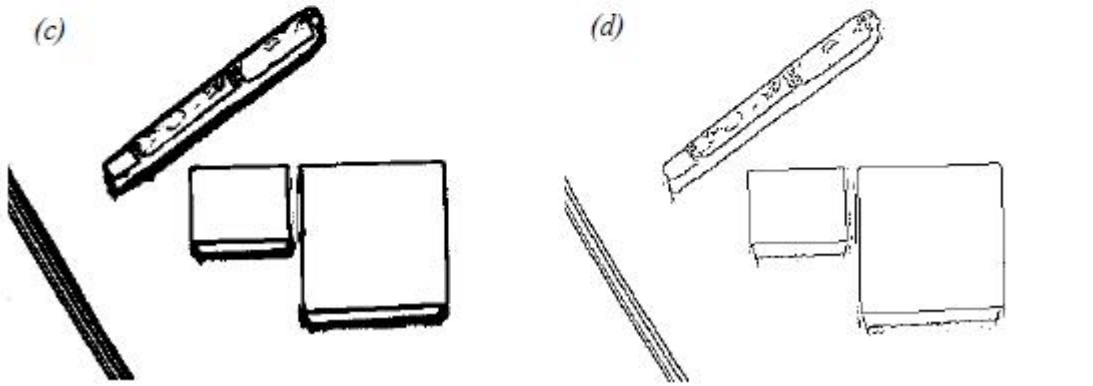
Thinning: non-maximal suppression
⇨ **edge image**

© M. Chli, P. Furgale, M. Hutter, M. Rufli, D. Scaramuzza, R. Siegwart, Autonomous Mobile Robots, ETH, 2016, http://www.asl.ethz.ch/education/lectures/autonomous_mobile_robots.html

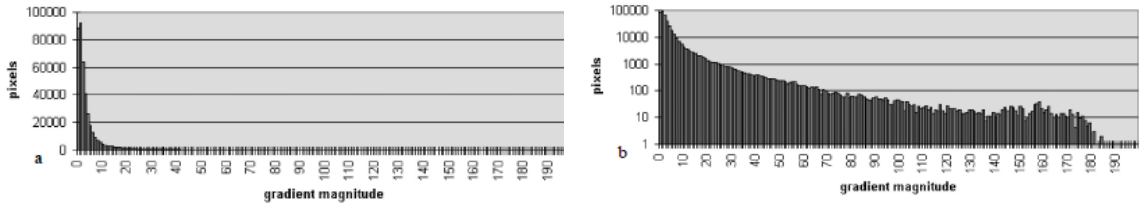
Dynamic thresholding

- Многие **алгоритмы обработки изображений проверяются** в **стабильных**, контролируемых **условиях** лабораторий **или на статических изображениях**. Это обеспечивает точность и повторяемость тестов, но не всегда отражает реалии работы в динамической среде.
- **Мобильные роботы в реальных условиях**: сталкиваются с меняющимися условиями окружающей среды, например, изменения освещения.
- **Необходимость адаптации**: системы зрения мобильных роботов должны быть способны адаптироваться к изменениям освещения. **Постоянный пороговый уровень для обнаружения краев** не всегда подходит.
- Один из подходов заключается в использовании **адаптивных порогов**, которые рассчитываются на основе **статистического анализа пикселей**.
- Для адаптации порога используется **гистограмма величин градиента**. Градиент отображает изменения интенсивности пикселей. Гистограмма показывает распределение этих изменений, что помогает **определить порог для обнаружения краев**. Это также позволяет **динамически подстраивать порог** в зависимости от реального распределения значений интенсивности на изображении.





Thresholding Nonmaxima suppression



Histogram of the gradient magnitudes: linear & log scales

Fig. © Roland Siegwart, Illah R. Nourbakhsh, and Davide Scaramuzza. Introduction to Autonomous Mobile Robots, MIT press, 2011

Recap: Blurring, Canny, Dilation and Erosion


```python
# Canny filter. Erosion and Dilation
import cv2
import numpy as np

img = cv2.imread("Resources/lena.png")
print(img.shape)
imgResize = cv2.resize(img, None, fx=0.5, fy=0.5, interpolation=cv2.INTER_LINEAR)

kernel = np.ones((3, 3), 'uint8')

imgGray = cv2.cvtColor(imgResize, cv2.COLOR_BGR2GRAY)
imgBlur = cv2.GaussianBlur(imgGray,(7,7),0)
imgCanny = cv2.Canny(imgBlur,100,100)
imgCanny2 = cv2.Canny(imgResize,100,100)
imgDilation = cv2.dilate(imgCanny, kernel=kernel, iterations=1)
imgErosion = cv2.erode(imgDilation, kernel=kernel, iterations=1)
imgDilation2 = cv2.dilate(imgGray, kernel=kernel, iterations=1)
imgErosion2 = cv2.erode(imgDilation2, kernel=kernel, iterations=1)

cv2.imshow("Gray Image", imgGray)
cv2.imshow("Blurred Image", imgBlur)
cv2.imshow("Canny Image", imgCanny)
cv2.imshow("Canny Image 2", imgCanny2)
cv2.imshow("Dilated Image", imgDilation)
cv2.imshow("Eroded Image", imgErosion)
cv2.imshow("Dilated Image 2", imgDilation2)
cv2.imshow("Eroded Image 2", imgErosion2)

cv2.waitKey(0)
```



- Дилатация и эрозия (Dilation and Erosion) — это морфологические операции, которые используются одна за другой для удаления мелких объектов на изображении.
- Фильтр Кэнни обычно применяют после размытия, которое удаляет шум.

Function	Description
cv2.Canny()	applies Canny Edge Detection with gradient search of smoothed image, discarding pixels below a certain threshold, and identifying local maxima with non-maxima suppression
cv2.dilate()	convolves an image with a kernel, adding pixels to object's boundaries in image
cv2.erode()	convolves an image with a kernel, removing pixels on object's boundaries in image

© собственная иллюстрация

2D edge detection: other approx. of derivative filters

- Prewitt:

$$F_x = \begin{bmatrix} -1 & 0 & 1 \\ -1 & 0 & 1 \\ -1 & 0 & 1 \end{bmatrix} \qquad F_y = \begin{bmatrix} 1 & 1 & 1 \\ 0 & 0 & 0 \\ -1 & -1 & -1 \end{bmatrix}$$

- Sobel:

$$F_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix} \qquad F_y = \begin{bmatrix} 1 & 2 & 1 \\ 0 & 0 & 0 \\ -1 & -2 & -1 \end{bmatrix}$$

- Roberts:

$$F_x = \begin{bmatrix} 0 & 1 \\ -1 & 0 \end{bmatrix} \qquad F_y = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}$$

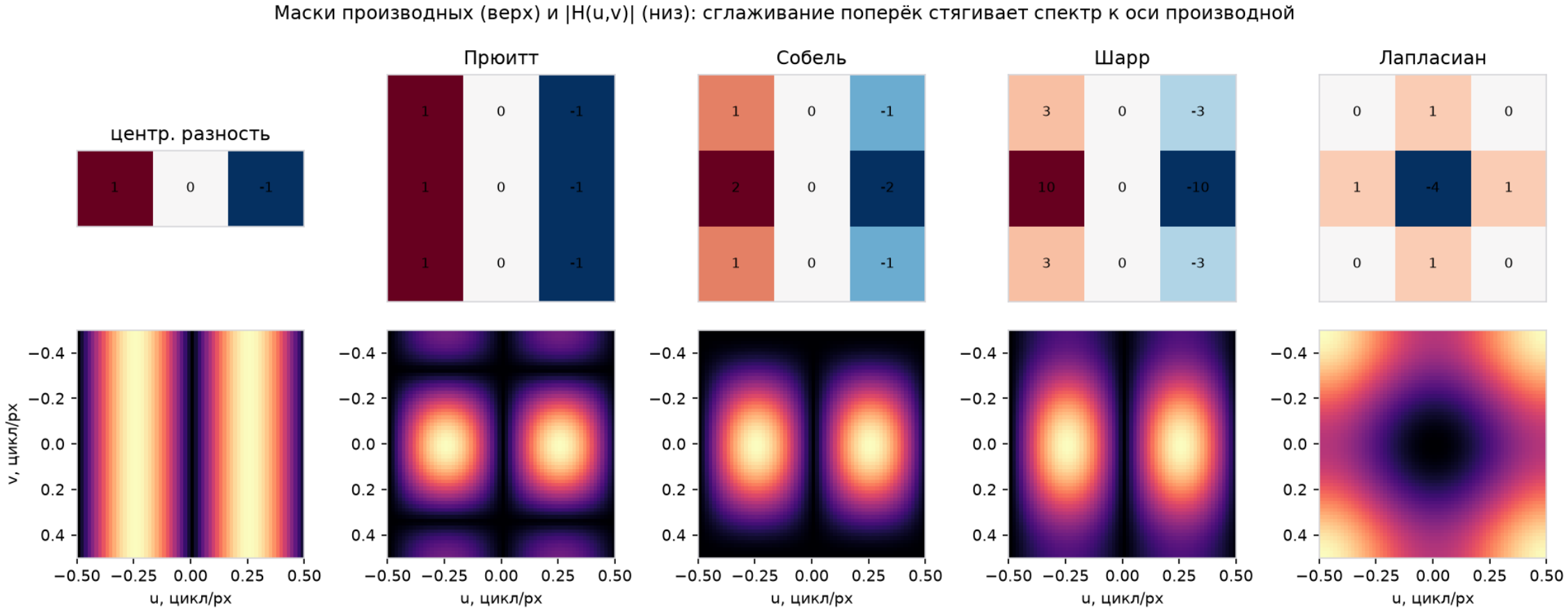
Sample Matlab code

```
>> im = imread('lion.jpg')
>> Fy = fspecial('sobel');
>> outim = imfilter(double(im), Fy);
>> imagesc(outim);
>> colormap gray;
```



© M. Chli, P. Furgale, M. Hutter, M. Rufli, D. Scaramuzza, R. Siegwart, Autonomous Mobile Robots, ETH, 2016, http://www.asl.ethz.ch/education/lectures/autonomous_mobile_robots.html

Маски: Собель как пара свёрток

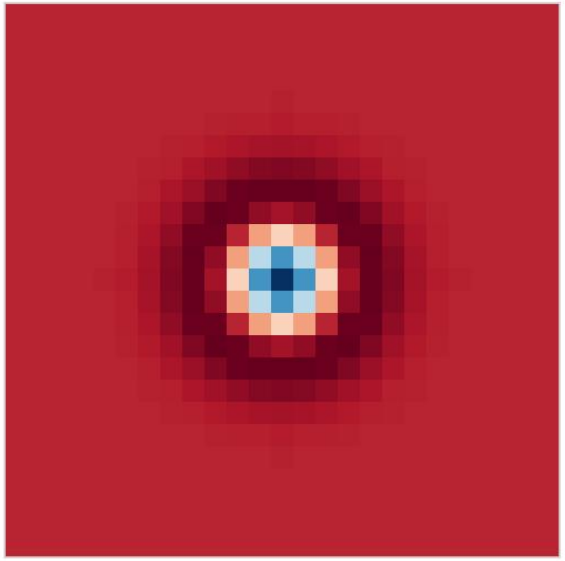


Двумерная маска Собеля — это **сепарабельный фильтр**. Ее можно разложить на последовательное применение двух одномерных сверток: одна вычисляет разность (производную), а вторая выполняет сглаживание по другой оси для подавления высокочастотного шума.

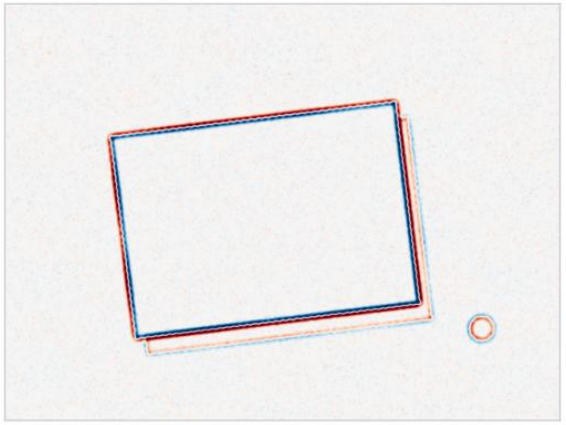
© собственная иллюстрация

Вторая производная: нуль-переходы

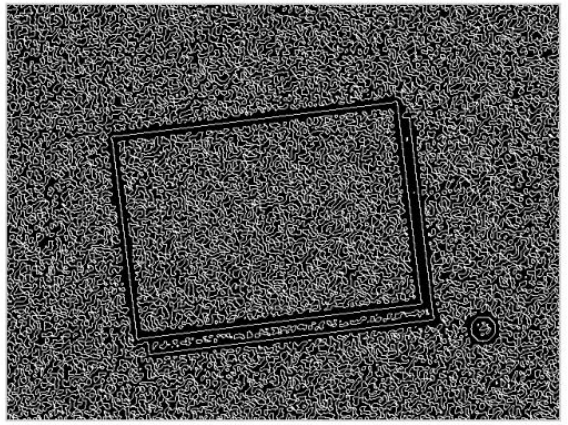
ядро LoG, σ=2 («мексиканская шляпа»)



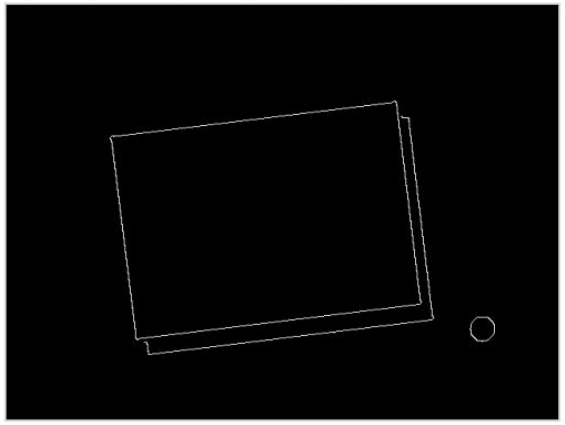
$\nabla^2(G*I)$: знак меняется на крае



все нуль-переходы: контуры замкнуты, но кольца



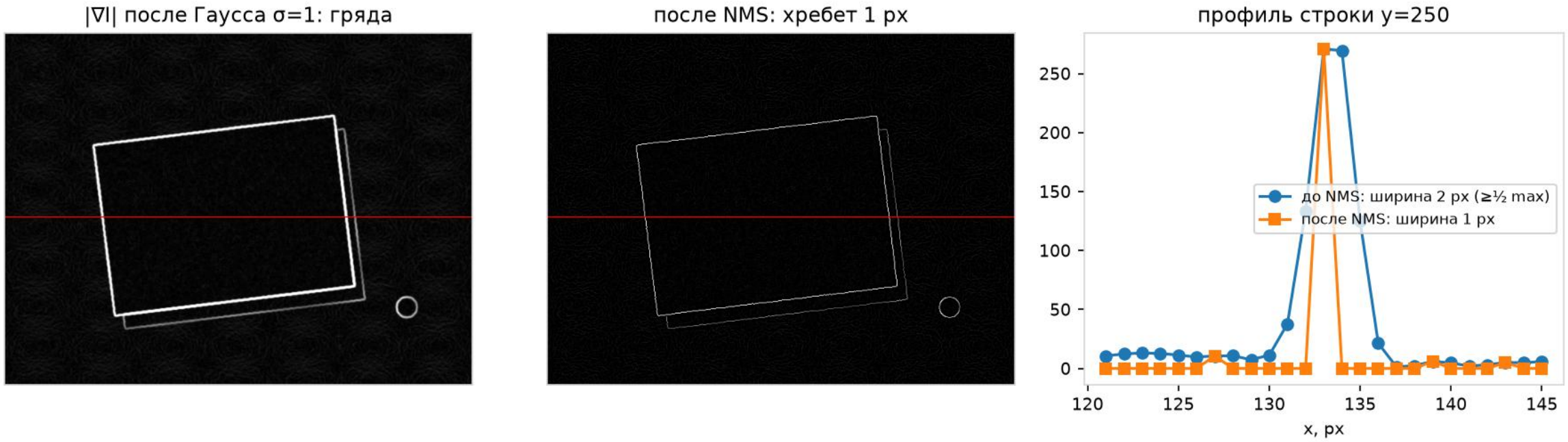
нуль-переходы при $|\nabla(G*I)|>25$



Использование второй производной для поиска точных границ с помощью фильтра **LoG (Laplacian of Gaussian)**. Вместо поиска максимумов первой производной границы определяются по точкам **перехода через ноль (zero-crossings)**. Это позволяет получить замкнутые контуры, а предварительное размытие по Гауссу убирает ложные срабатывания от шума.

© собственная иллюстрация

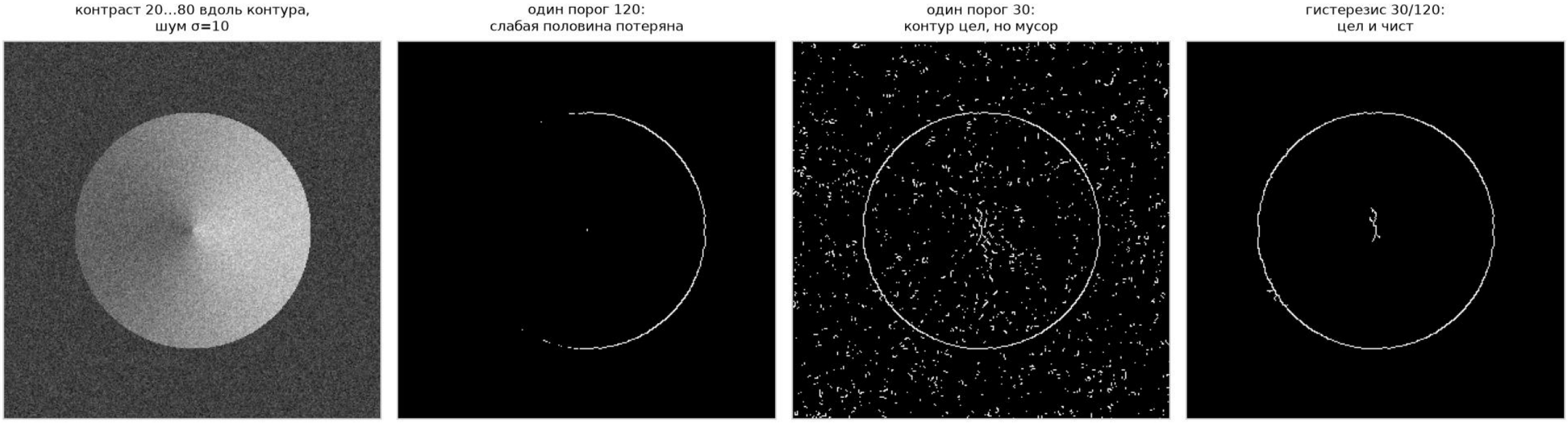
NMS: край толщиной в один пиксель



Применение алгоритма **Non-Maximum Suppression (подавление немаксимумов)** для истончения размытых границ. Метод оставляет только пиксели с максимальным значением градиента вдоль направления границы (так называемый «хребет»), превращая широкую полосу в четкую линию толщиной в **один пиксель**.

© собственная иллюстрация

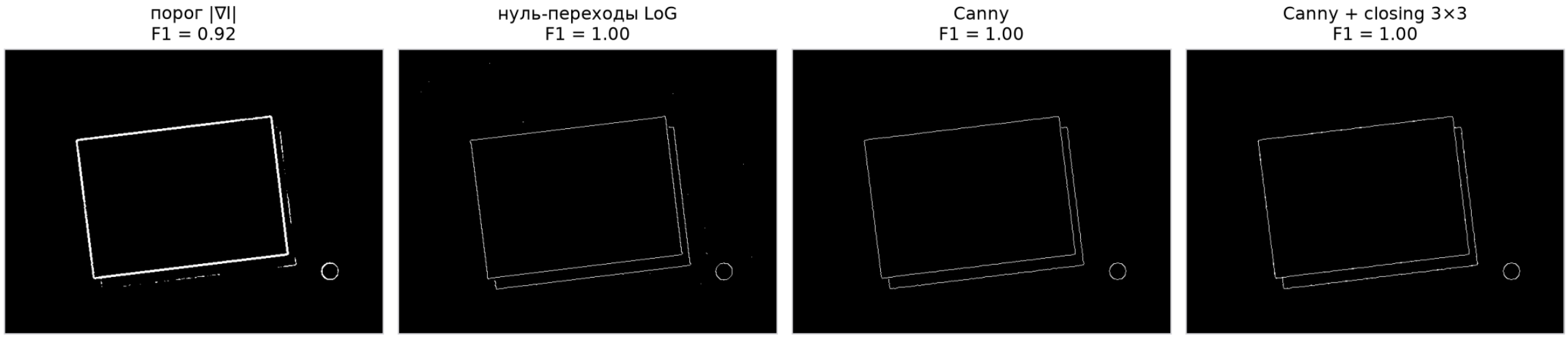
Гистерезис: два порога и связность



Решение проблемы выбора идеального порога для фильтрации шума. Вместо одного порога используются **два (верхний и нижний)**. Верхний порог находит гарантированные (сильные) границы, а нижний позволяет сохранить слабые границы, но только если они физически связаны с сильными. Это убирает изолированный шум, сохраняя целостность контура.

© собственная иллюстрация

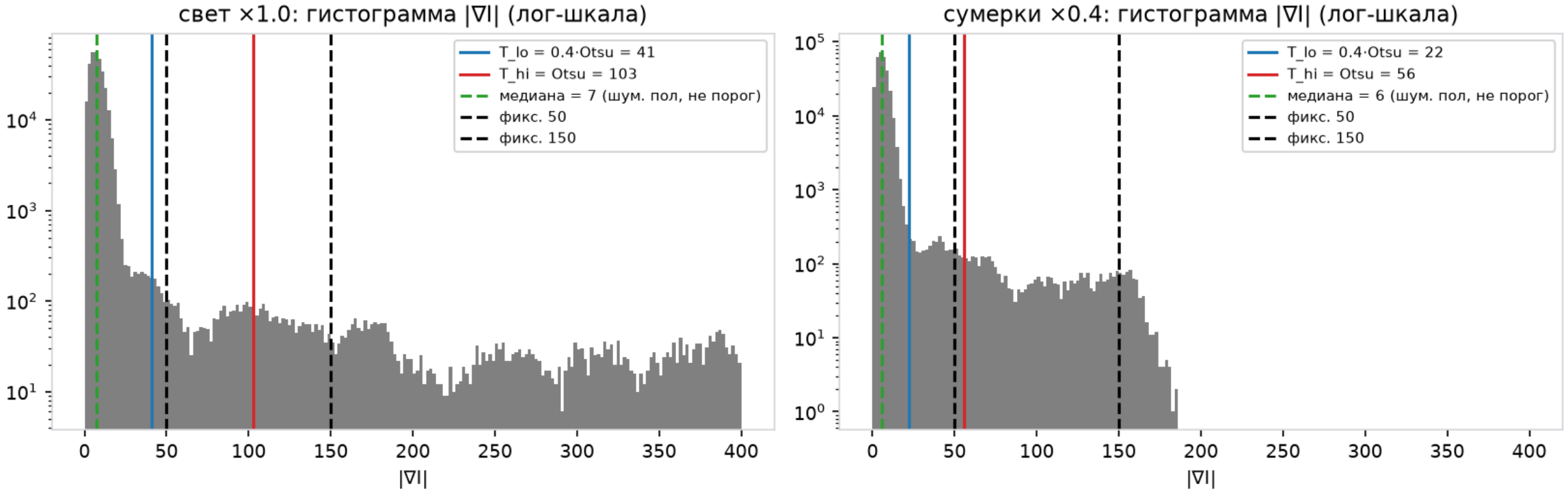
Четыре детектора на одном кадре



Сравнительный анализ качества работы разных методов выделения границ на одном изображении. Простой порог градиента или нуль-переходы LoG дают много шума или разрывов, в то время как **детектор Кэнни (Canny)** и его комбинация с морфологическим закрытием (closing) дают наиболее чистый и замкнутый контур.

© собственная иллюстрация

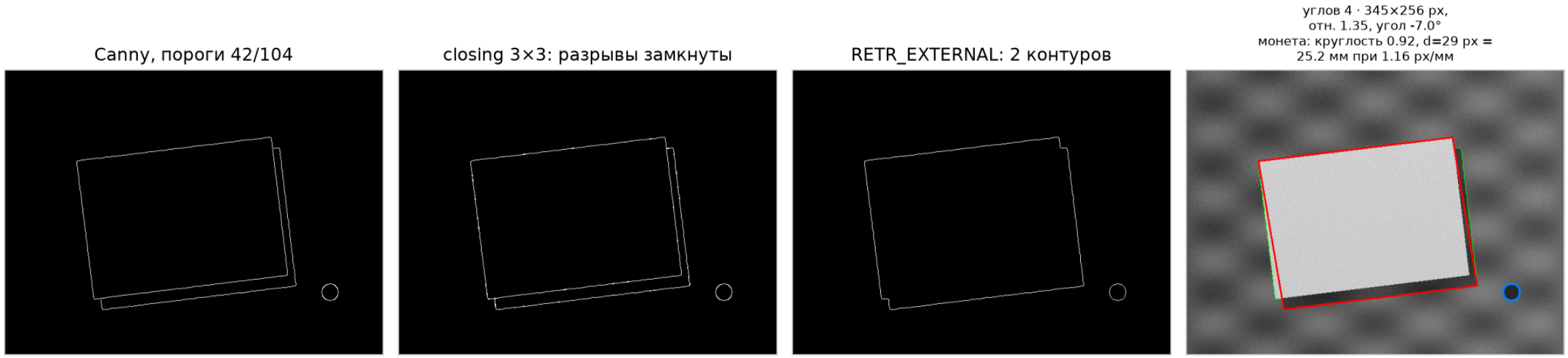
Динамические пороги: гистограмма градиента



Автоматизация выбора порогов для детектора границ в зависимости от освещения (день/сумерки). Вместо жестко заданных констант предлагается анализировать **гистограмму распределения градиентов** кадра (в логарифмической шкале) и динамически вычислять пороговые значения для адаптивной работы алгоритма.

© собственная иллюстрация

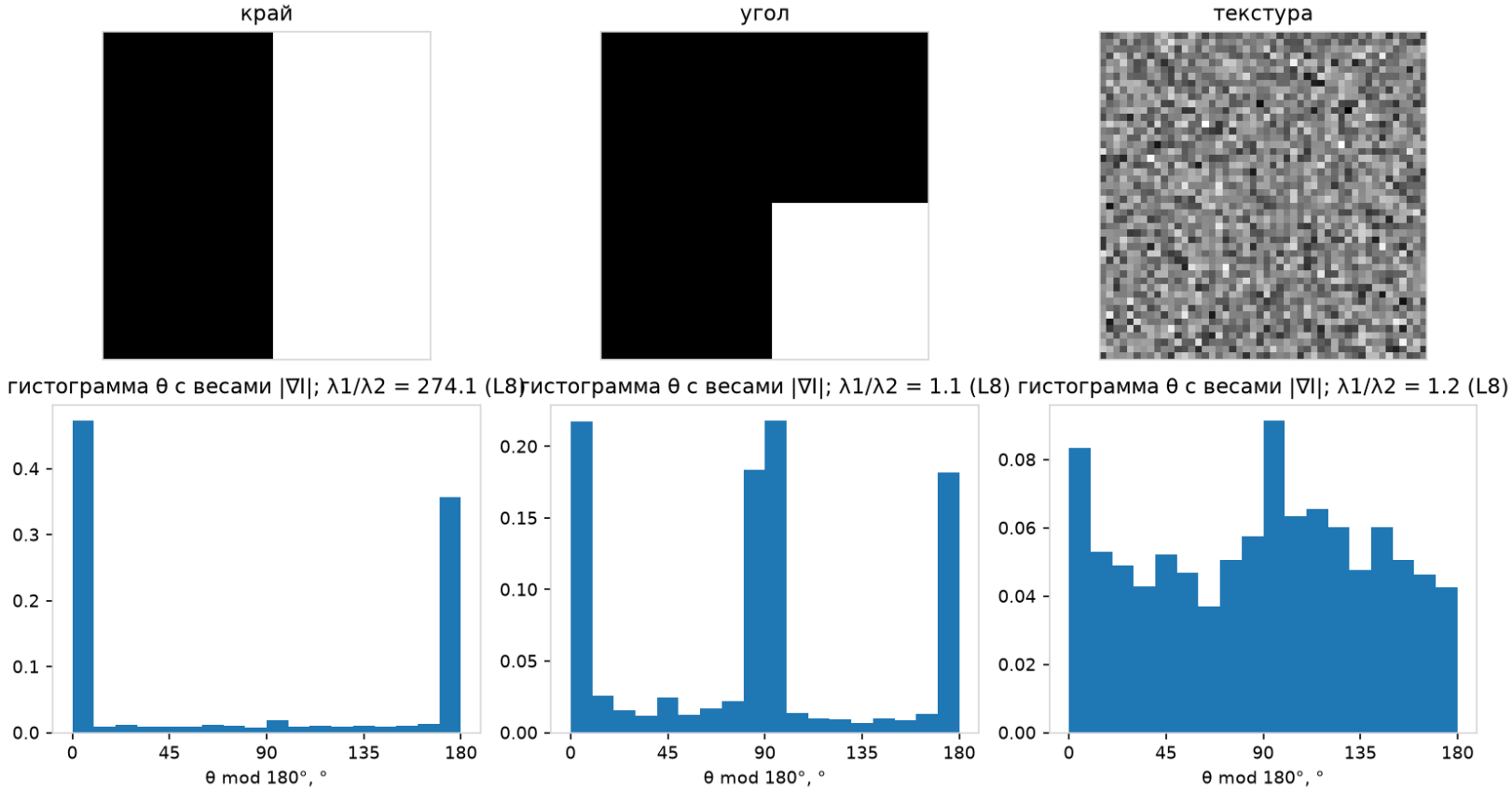
Контуры: морфология и findContours



Переход от бинарной маски пикселей к структурному описанию объекта. Сначала с помощью морфологической операции closing устраняются мелкие разрывы в границах Кэнни, а затем функция **findContours** (из OpenCV) объединяет пиксели в связанные векторные контуры (например, для поиска углов и аппроксимации геометрии прямоугольника).

© собственная иллюстрация

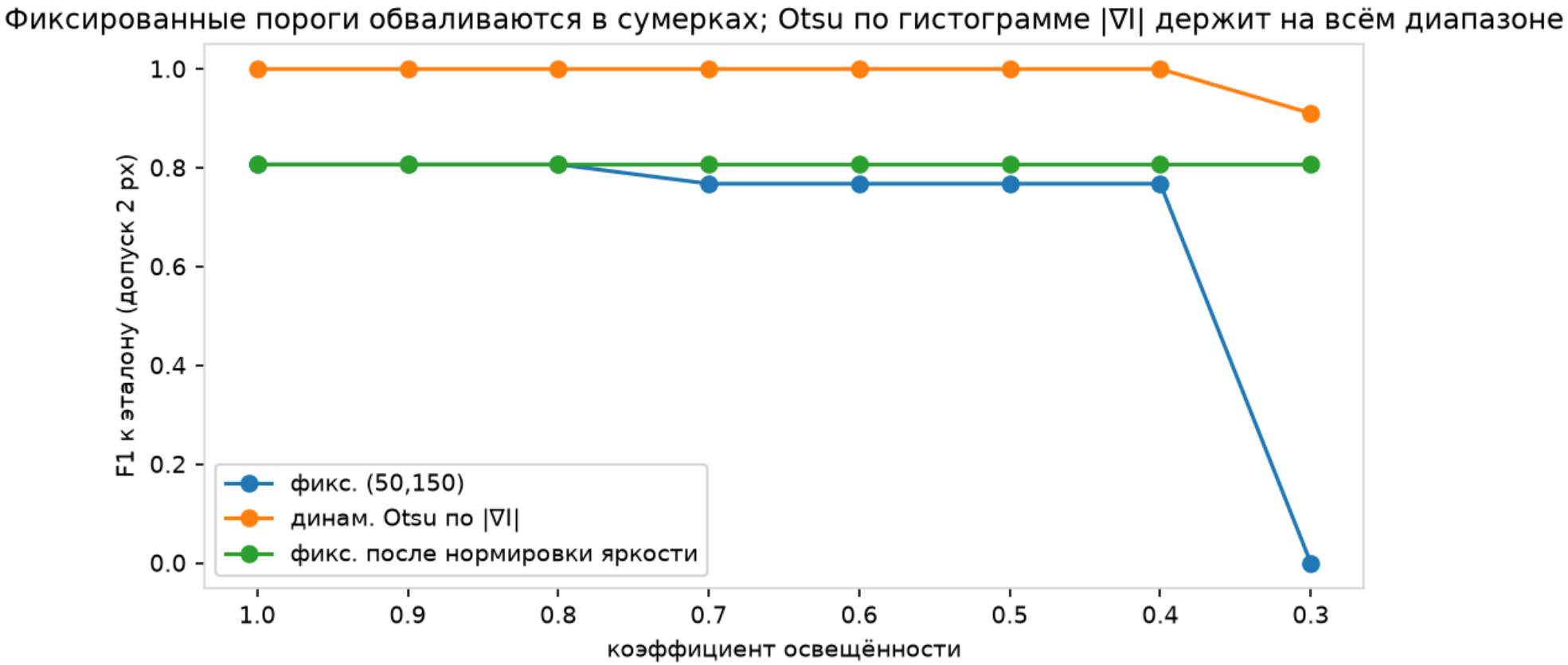
Ориентация градиента: мост к L8 и L9



Использование гистограмм ориентации градиентов для автоматического распознавания типа локальной структуры. По форме гистограммы направлений (в пределах от 0° до 180°) можно четко разделить геометрические примитивы: **край** дает один ярко выраженный пик, **угол** формирует два четких пика (соответствующих двум перпендикулярным границам), а случайная **текстура** или шум дают хаотичное и равномерное распределение углов.

© собственная иллюстрация

Измерение: пороги против освещения



Экспериментальное доказательство неэффективности жестких (фиксированных) порогов бинаризации при ухудшении видимости и превосходство адаптивных методов. При падении коэффициента освещенности (в сумерках) качество работы алгоритма с фиксированными порогами стремительно падает до нуля. При этом динамический расчет порогов методом **Отсу (Otsu)** по гистограмме градиента или предварительная **нормировка яркости** позволяют сохранять стабильно высокое качество обнаружения (метрика F1 близка к максимуму) во всем диапазоне освещенности.

© собственная иллюстрация

Итог части 1

- Умеем: градиент и направление (Scharr 0.31° против Собеля 4.89°); SNR края 3.3 → 5.0 → 7.8 после Гаусса; Canny вручную совпадает с cv2.Canny на F1 1.0; контур листа → 4 угла и размеры
- Не умеем: описать край целиком набором пикселей — пиксели не знают, что они одна прямая; шумные точки голосуют наравне с настоящими
- Ответ — параметризация прямой и голосование: преобразование Хафа

Демо 1.1 — градиент и цена шума

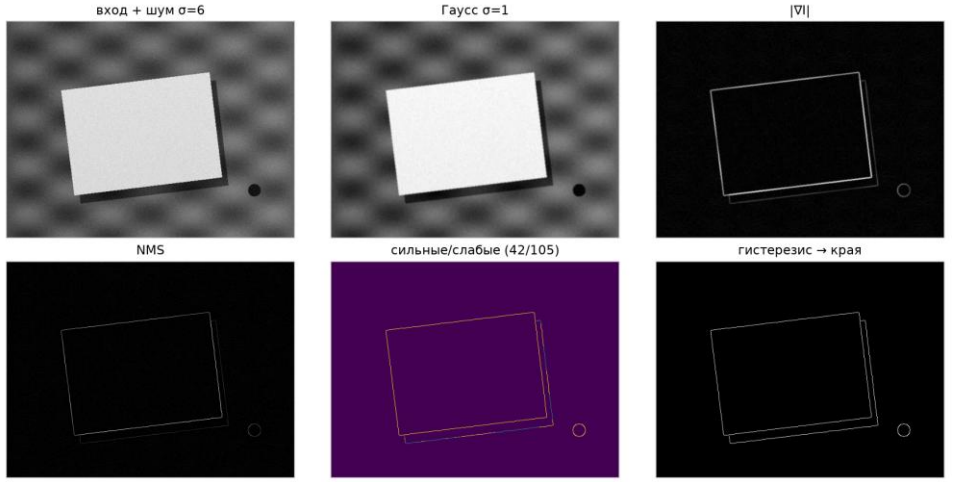

```python
n = cio.imread("demo/iu-building.jpg", "gray").astype(np.float64)
n += np.random.default_rng(0).normal(0, 10, n.shape)       # шум сенсора σ=10
sob = lambda im, k: cv2.Sobel(im, cv2.CV_64F, 1, 0, ksize=k)
mag = lambda im, k: cv2.magnitude(sob(im, k), cv2.Sobel(im, cv2.CV_64F, 0, 1, ksize=k))
flat = (slice(700, 900), slice(300, 600))                  # однородный участок
print("std |∇| на плоском: центр. разность %.1f · Собель %.1f · Собель после G σ1 %.1f"
      % (mag(n, 1)[flat].std(), mag(n, 3)[flat].std(),
         mag(cv2.GaussianBlur(n, (0, 0), 1.0), 3)[flat].std()))
print("пик края: 25 / 16 / 9.4 · SNR 3.3 / 5.0 / 7.8")
```

std |∇| на плоском: центр. разность 7.5 · Собель 34.6 · Собель после G σ1 12.4
пик края: 25 / 16 / 9.4 · SNR 3.3 / 5.0 / 7.8 (теория ‖h‖σ — те же числа с fig03)

Вызов	Что делает
`cv2.Sobel(im, CV_64F, 1, 0, ksize)`	производная по x; ksize=1 — центральная разность без сглаживания
`cv2.magnitude(gx, gy)`	модуль градиента; `cv2.phase(gx, gy, True)` — его направление в градусах

Демо 1.2 — Canny вручную против cv2.Canny


```python
def canny_manual(im, sigma, lo, hi):
    s = cv2.GaussianBlur(im, (0, 0), sigma)
    gx = cv2.Sobel(s, cv2.CV_64F, 1, 0); gy = cv2.Sobel(s, cv2.CV_64F, 0, 1)
    mag, ang = cv2.magnitude(gx, gy), cv2.phase(gx, gy, True)
    q = (np.round(ang / 45).astype(int) % 4)              # 0,45,90,135°
    nm = mag * ((mag >= shift(mag, q, 1)) & (mag >= shift(mag, q, -1)))
    strong, weak = nm >= hi, (nm >= lo) & (nm < hi)
    lab, n = cv2.connectedComponents((strong | weak).astype(np.uint8), 8)
    keep = np.zeros(n + 1, bool); keep[np.unique(lab[strong])] = True
    return (keep[lab] & (strong | weak)).astype(np.uint8) * 255
```



Вызов	Что делает
`shift(mag, q, ±1)`	соседи вдоль квантованного направления градиента: NMS сравнивает только с ними
`cv2.connectedComponents`	гистерезис без рекурсии: слабые остаются, если в их компоненте есть сильный

© собственная иллюстрация

Пауза: два вопроса

- Вопрос 1. Почему Собель сглаживает именно поперёк производной, и что станет с его спектром, если сглаживание убрать? (A ничего · B спектр растянется вдоль оси производной · C появятся лепестки по обеим осям · D спектр станет круглым)
- Вопрос 2. Кадр стал вдвое темнее и вдвое малоконтрастнее. σ Гаусса и пороги Canny: что менять? (A ничего · B только пороги ×0.5 · C только σ ×2 · D пороги ×0.5 и σ ×2)

2

Прямые: преобразование Хафа

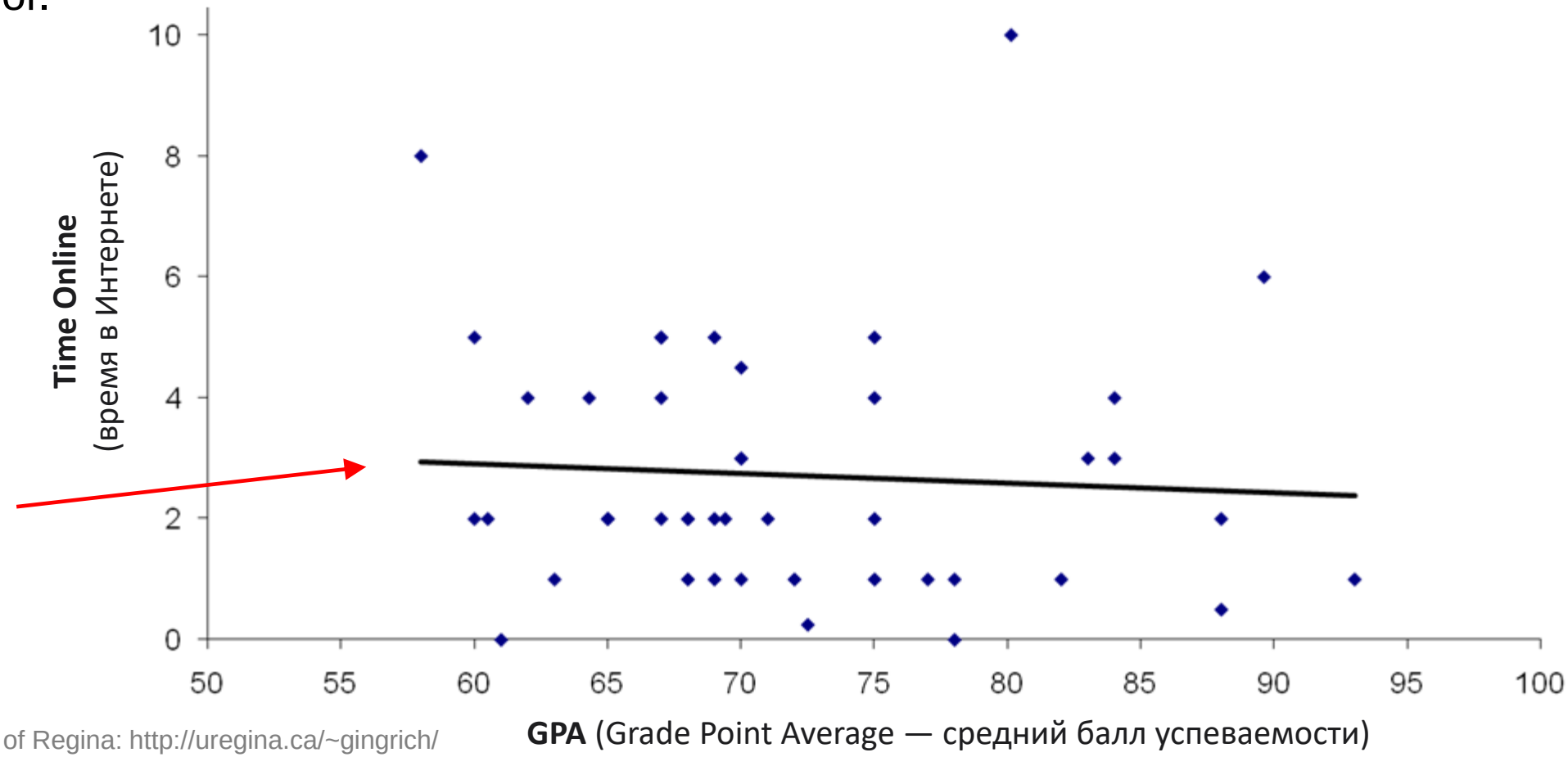
Дуальность · аккумулятор · квантование · HoughLinesP · дубли и кластеризация · вырожденные случаи

Outliers (1/3)

- Rare, extreme values may distort the outcome.
  - Could be an important observation.
  - Could be an error.

Outlier: more than 2-3 standard deviations from the mean.

Linear regression

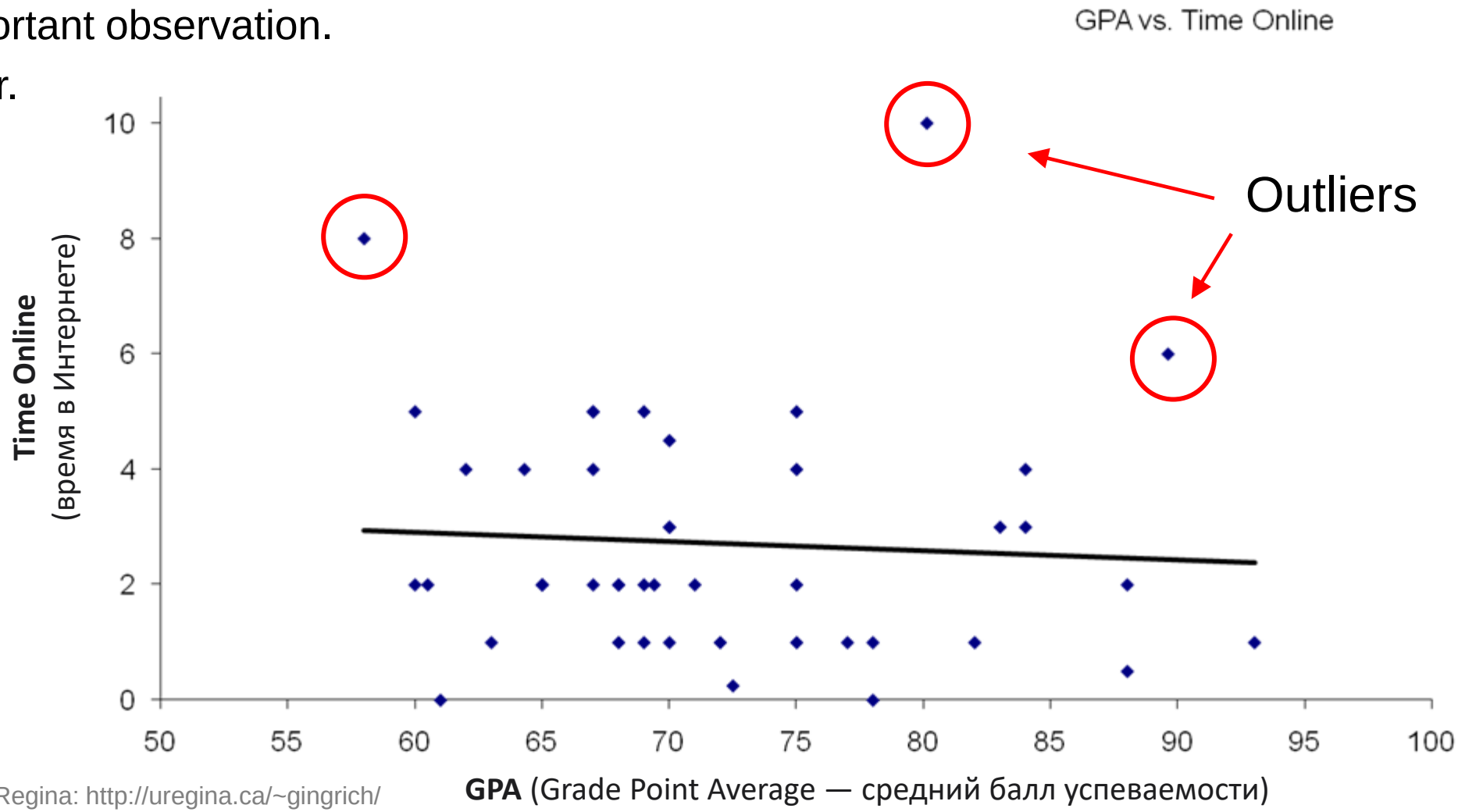


(c) Web site of Paul Gingrich, University of Regina: http://uregina.ca/~gingrich/

Outliers (2/3)

- Rare, extreme values may distort the outcome.
  - Could be an important observation.
  - Could be an error.

Outlier: more than 2-3 standard deviations from the mean.



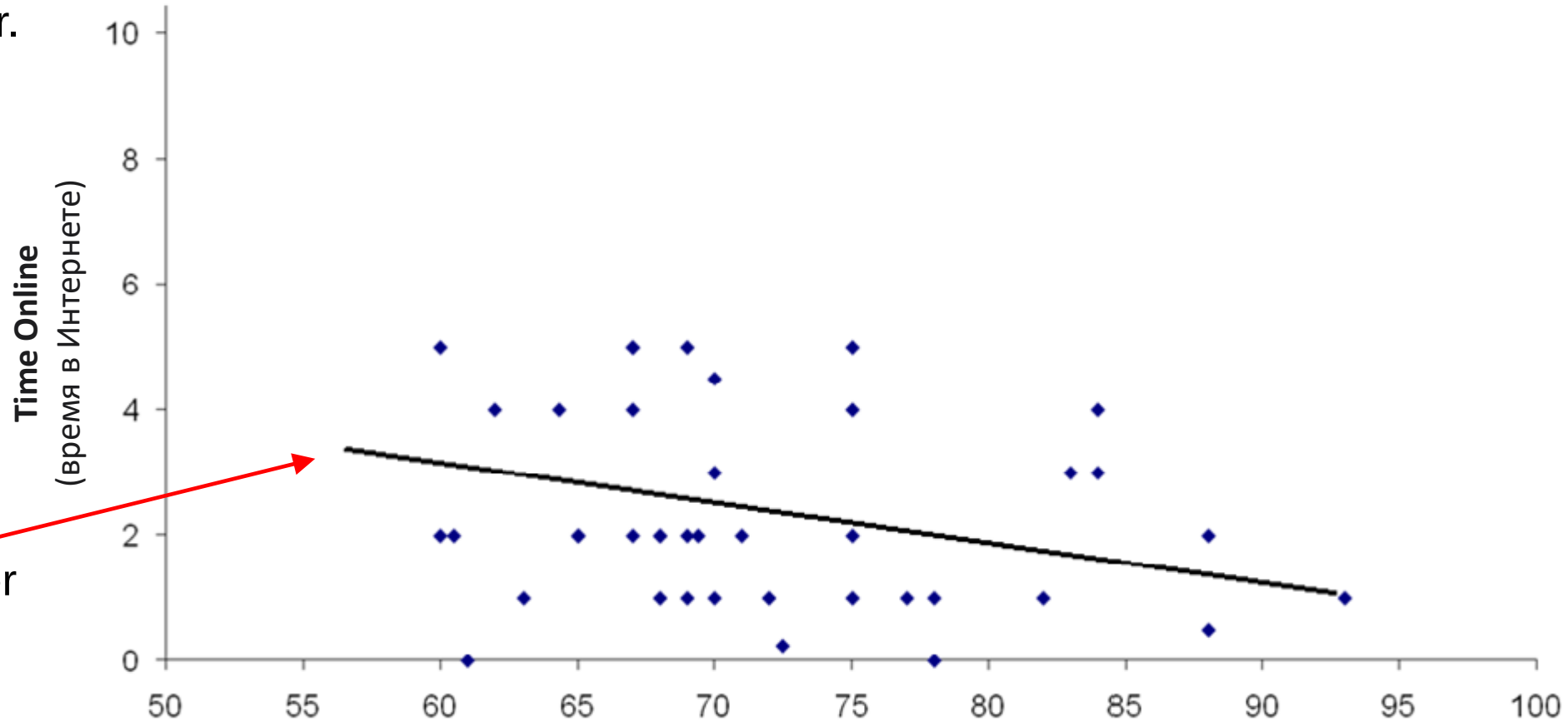
(c) Web site of Paul Gingrich, University of Regina: http://uregina.ca/~gingrich/

Outliers (3/3)

- Rare, extreme values may distort the outcome.
  - Could be an important observation.
  - Could be an error.

Outlier: more than 2-3 standard deviations from the mean.

New linear regression after outliers' elimination

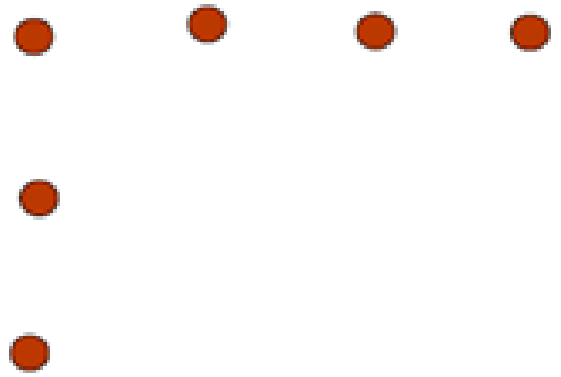


(c) Web site of Paul Gingrich, University of Regina: http://uregina.ca/~gingrich/

Line extraction: 1. split-and-merge (standard)

- **Алгоритм разбиения и слияния (split-and-merge)** для извлечения линий из набора точек
  - Популярный алгоритм из области **компьютерного зрения**
  - Используется для аппроксимации набора точек ломаной линией
- **Принцип работы**:

  Это рекурсивная процедура, состоящая из двух этапов:
  - **Split (разбиение**) — разделение набора точек на сегменты
  - **Merge (слияние)** — объединение соседних сегментов, которые могут быть аппроксимированы одной прямой
- **Вариант работы алгоритма**:
  1. Соединяем все точки одной линией (от первой до последней)
  2. Находим точку с **максимальным отклонением** от этой линии
  3. Если отклонение > порога → **разбиваем** сегмент на два в этой точке
  4. Рекурсивно повторяем для каждого нового сегмента
  5. Затем пытаемся **объединить** соседние сегменты, если они коллинеарны



© M. Chli, P. Furgale, M. Hutter, M. Rufli, D. Scaramuzza, R. Siegwart, Autonomous Mobile Robots, ETH, 2016

Line extraction: 1. split-and-merge (standard)

- Popular algorithm, originates from Computer Vision.
- A recursive procedure of fitting and splitting.
- A slightly different version, called Iterative end-point-fit, simply connects the end points for line fitting.

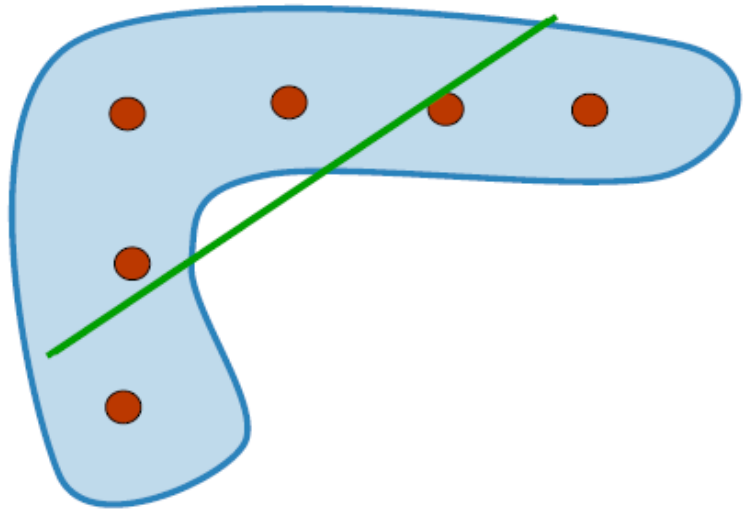
Let **S** be the set of all data points

Split

- Fit a line to points in current set **S**
- Find the most distant point to the line
- If distance > threshold ⇨ split set & repeat with left and right point sets

Merge

- If two consecutive segments are collinear enough, obtain the common line and find the most distant point
- If distance <= threshold, merge both segments



© M. Chli, P. Furgale, M. Hutter, M. Rufli, D. Scaramuzza, R. Siegwart, Autonomous Mobile Robots, ETH, 2016

Line extraction: 1. split-and-merge (standard)

- Popular algorithm, originates from Computer Vision.
- A recursive procedure of fitting and splitting.
- A slightly different version, called Iterative end-point-fit, simply connects the end points for line fitting.

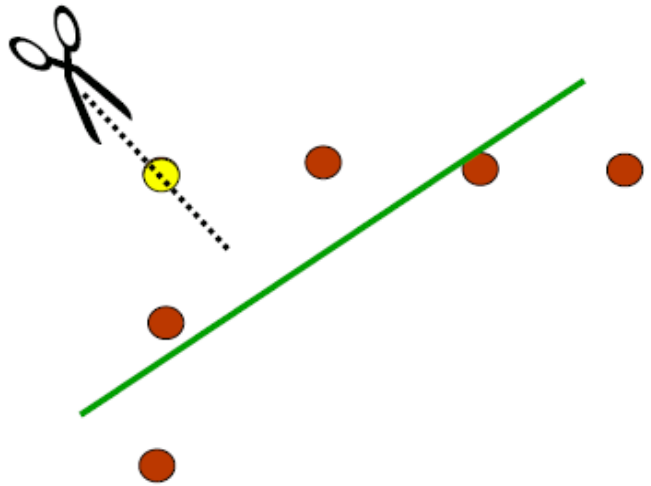
Let **S** be the set of all data points

Split

- Fit a line to points in current set **S**
- Find the most distant point to the line
- If distance > threshold ⇨ split set & repeat with left and right point sets

Merge

- If two consecutive segments are collinear enough, obtain the common line and find the most distant point
- If distance <= threshold, merge both segments



© M. Chli, P. Furgale, M. Hutter, M. Rufli, D. Scaramuzza, R. Siegwart, Autonomous Mobile Robots, ETH, 2016

Line extraction: 1. split-and-merge (standard)

- Popular algorithm, originates from Computer Vision.
- A recursive procedure of fitting and splitting.
- A slightly different version, called Iterative end-point-fit, simply connects the end points for line fitting.

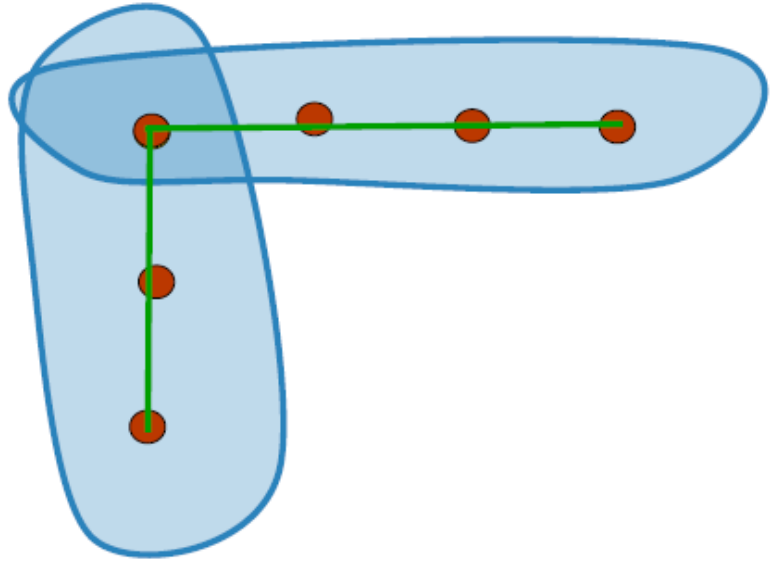
Let **S** be the set of all data points

Split

- Fit a line to points in current set **S**
- Find the most distant point to the line
- If distance > threshold ⇨ split set & repeat with left and right point sets

Merge

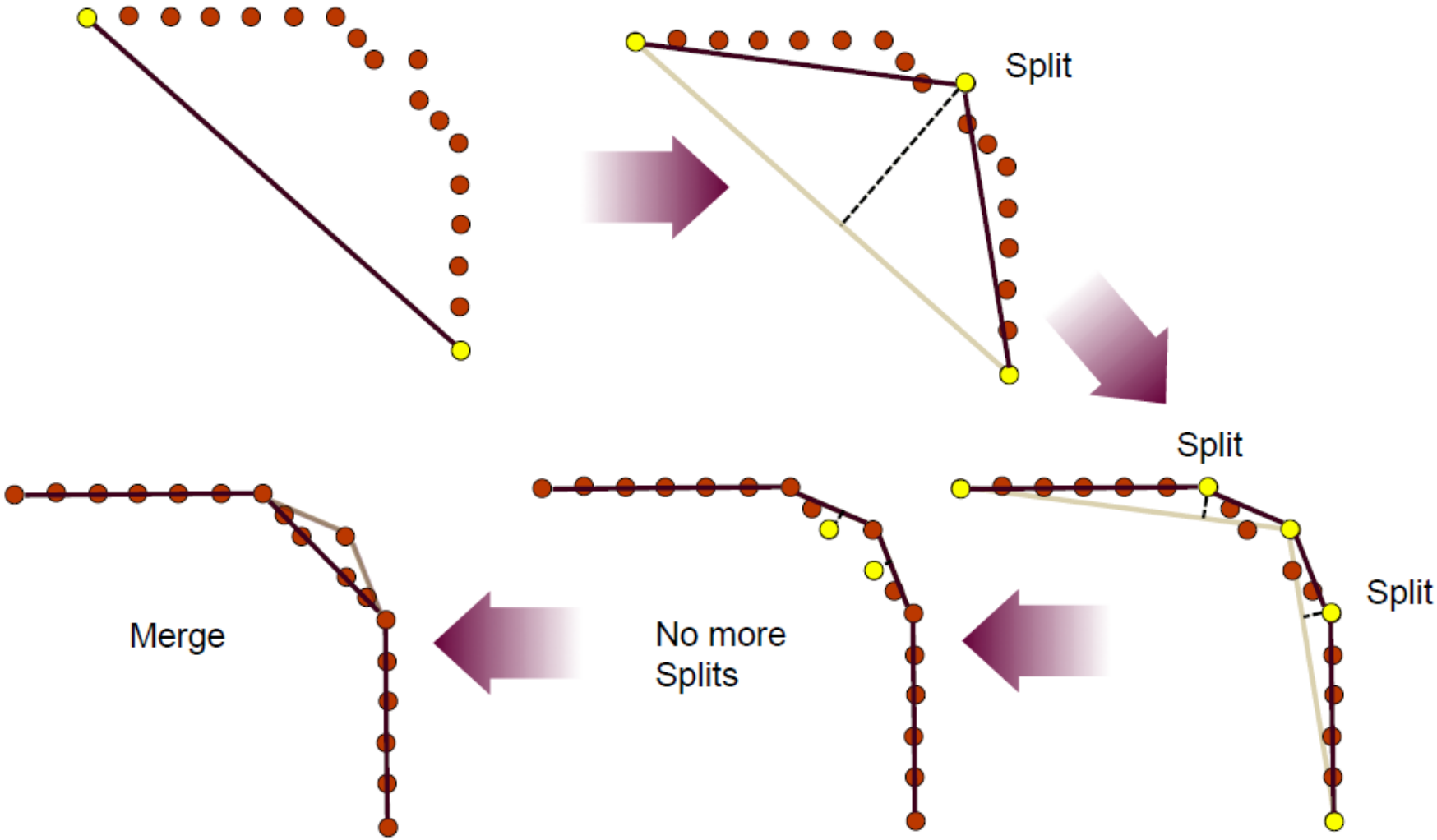
- If two consecutive segments are collinear enough, obtain the common line and find the most distant point
- If distance <= threshold, merge both segments



© M. Chli, P. Furgale, M. Hutter, M. Rufli, D. Scaramuzza, R. Siegwart, Autonomous Mobile Robots, ETH, 2016

Line extraction: 1. split-and-merge (end-point-fit)

Iterative end-point-fit: simply connects the end points for line fitting



После того как все наборы (сегменты) в ***L*** были разделены, можно объединить **коллинеарные сегменты**

© M. Chli, P. Furgale, M. Hutter, M. Rufli, D. Scaramuzza, R. Siegwart, Autonomous Mobile Robots, ETH, 2016

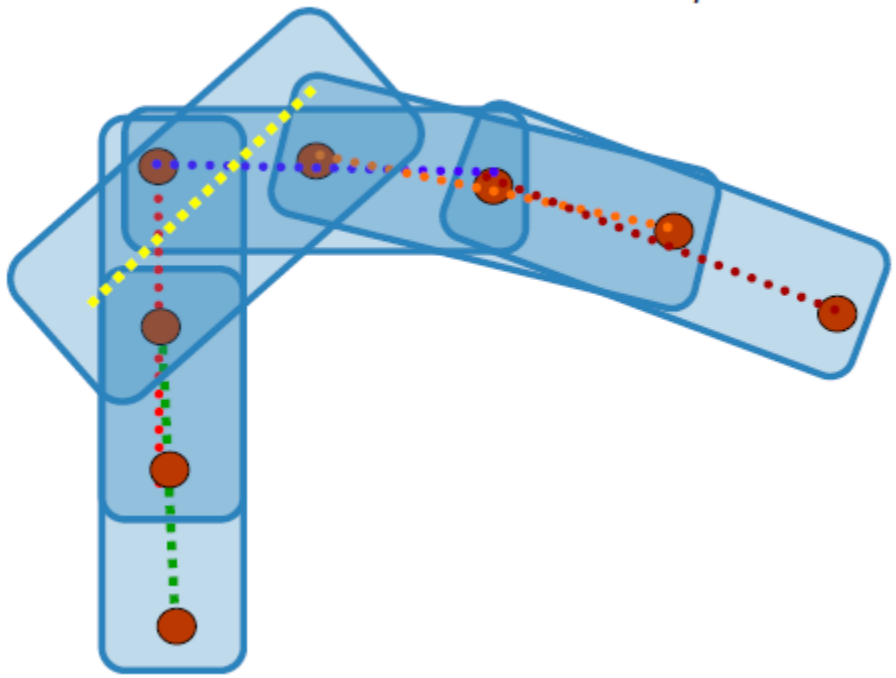
Line extraction: 2. line-regression

- "Sliding window" of size N_f points
- Fit line-segment to all points in each window

Line-Regression

- Initialize sliding window size N_f
- Fit a line to every N_f consecutive points (i.e. in each window)
- Merge overlapping line segments + re-compute line parameters for each segment

$N_f = 3$



- Алгоритм использует скользящее окно размером N_f. На каждом шаге линия подгоняется к точкам внутри окна. Затем окно сдвигается на одну точку вперед, и процесс подгонки линии повторяется снова.
- Цель заключается в том, чтобы найти смежные отрезки линии и объединить их вместе. Для этого на каждом шаге вычисляется расстояние Махаланобиса (**Mahalanobis distance**) между последними двумя окнами и сохраняется в массиве точности.

© M. Chli, P. Furgale, M. Hutter, M. Rufli, D. Scaramuzza, R. Siegwart, Autonomous Mobile Robots, ETH, 2016

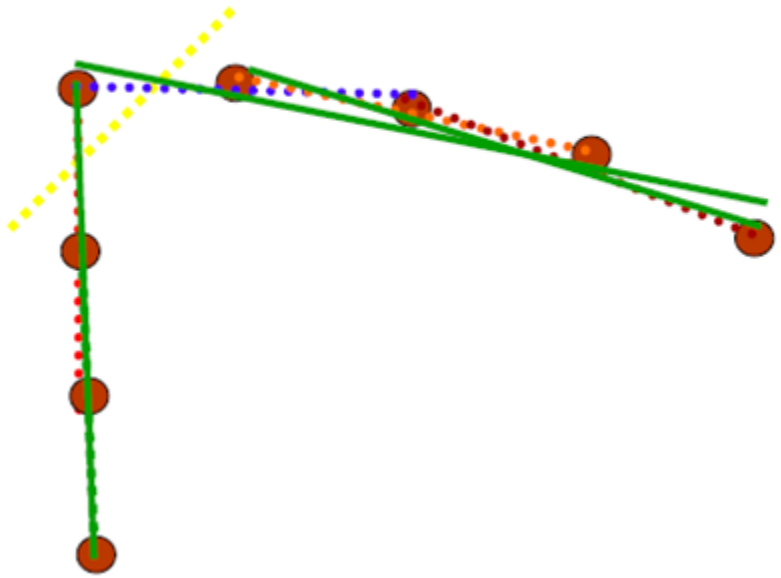
Line extraction: 2. line-regression

- "Sliding window" of size N_f points
- Fit line-segment to all points in each window

Line-Regression

- Initialize sliding window size N_f
- Fit a line to every N_f consecutive points (i.e. in each window)
- Merge overlapping line segments + re-compute line parameters for each segment

$N_f = 3$

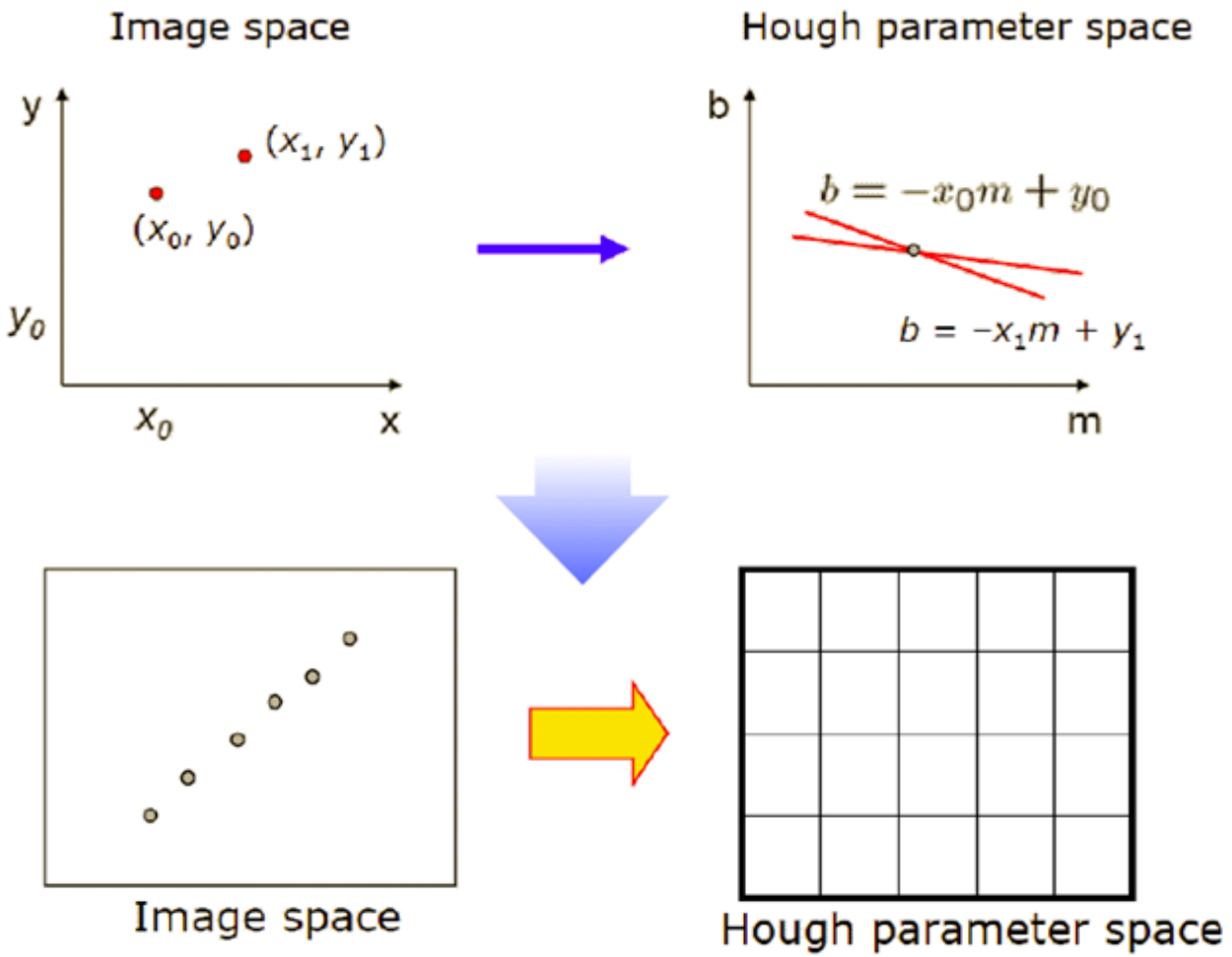


- ❑ Когда все точки были проанализированы, массив точности (fidelity array) сканируется на предмет последовательных аналогичных элементов. Это делается с помощью соответствующего алгоритма кластеризации.
- ❑ В конце сгруппированные последовательные линейные сегменты объединяются вместе с использованием **линейной регрессии**.
- ❑ Скользящее окно зависит от среды и влияет на производительность алгоритма. Обычно, ***N_f =7***

© M. Chli, P. Furgale, M. Hutter, M. Rufli, D. Scaramuzza, R. Siegwart, Autonomous Mobile Robots, ETH, 2016

3. Преобразование Хафа (Hough Transform)

- Преобразование Хафа переводит точки из **пространства изображения** (image space) в **пространство параметров** (parameter space), где линии представлены точками пересечения.
- Точки, лежащие на одной прямой в пространстве изображения, дают линии, которые **пересекаются в одной точке** в пространстве параметров Хафа
- Как это работает:
  - Для каждой точки изображения перебираем все возможные линии, проходящие через неё
  - Увеличиваем счётчики в соответствующих ячейках аккумулятора (пространство параметров)
  - Ячейки с максимальными значениями указывают на параметры обнаруженных линий
- **Преимущество:** Метод устойчив к разрывам в линиях и шуму

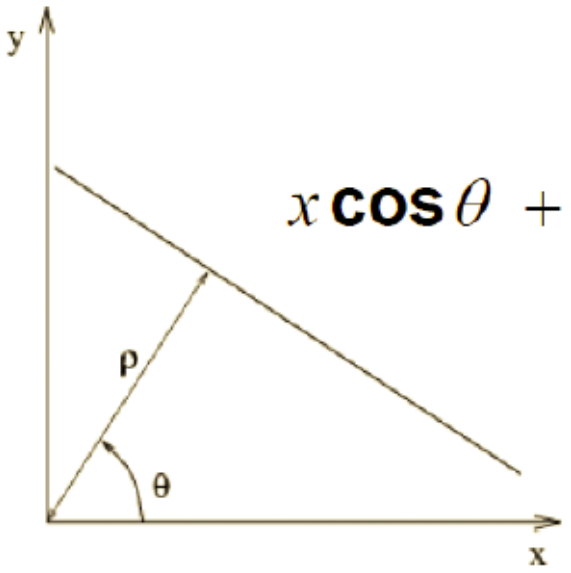


© M. Chli, P. Furgale, M. Hutter, M. Rufli, D. Scaramuzza, R. Siegwart, Autonomous Mobile Robots, ETH, 2016

Line extraction: 3. Hough Transform

- Problems with the (m,b) space:
  - Unbounded parameter domain
  - How to represent lines aligned with the axes?

- Alternative: polar representation

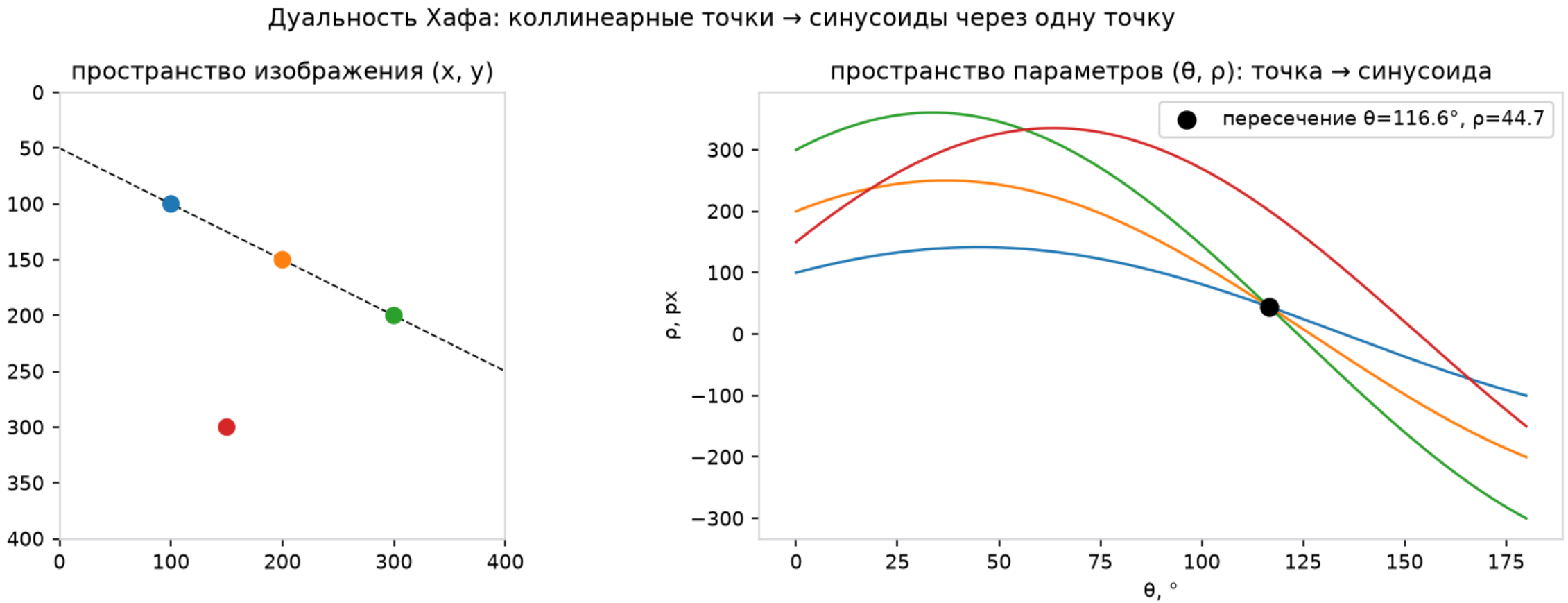


$$x\cos\theta + y\sin\theta = \rho$$

Each point in image space will map to a sinusoid in the (ρ,θ) parameter space

© M. Chli, P. Furgale, M. Hutter, M. Rufli, D. Scaramuzza, R. Siegwart, Autonomous Mobile Robots, ETH, 2016

Дуальность: точка ↔ прямая

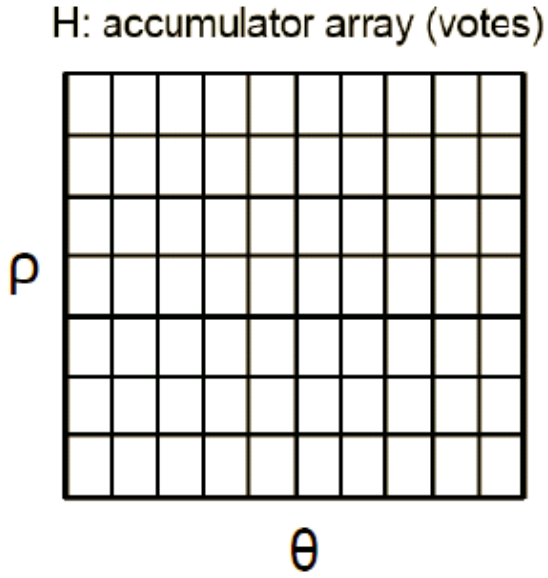


Коллинеарные точки (лежащие на одной прямой в пространстве изображений) преобразуются в синусоиды, которые **строго пересекаются в одной точке** в пространстве параметров. И наоборот: изолированная точка в пространстве изображений (например, красная точка-шум на графике) дает синусоиду, которая проходит мимо основного центра пересечения, что позволяет легко фильтровать подобные аномалии.

© собственная иллюстрация

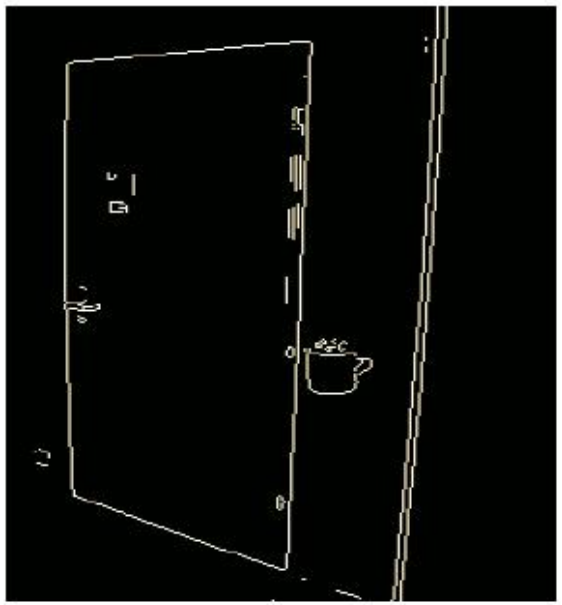
Line extraction: 3. Hough Transform

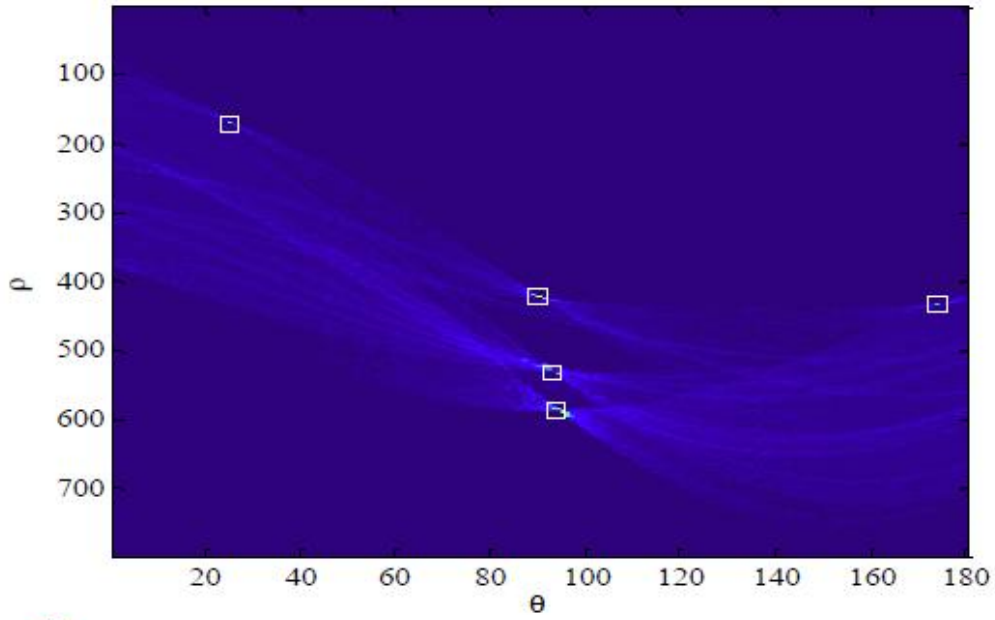
1. Initialize accumulator H to all zeros
2. **for** each edge point (x,y) in the image
   - **for** all θ in [0,180]
     - Compute $\rho = x \cos\theta + y \sin\theta$
     - $H(\theta, \rho) = H(\theta, \rho) + 1$
   - **end**
   - **end**
3. Find the values of (θ, ρ) where $H(\theta, \rho)$ is a local maximum
4. The detected line in the image is given by: $\rho = x \cos\theta + y \sin\theta$



© M. Chli, P. Furgale, M. Hutter, M. Rufli, D. Scaramuzza, R. Siegwart, Autonomous Mobile Robots, ETH, 2016

Line extraction: 3. Hough Transform



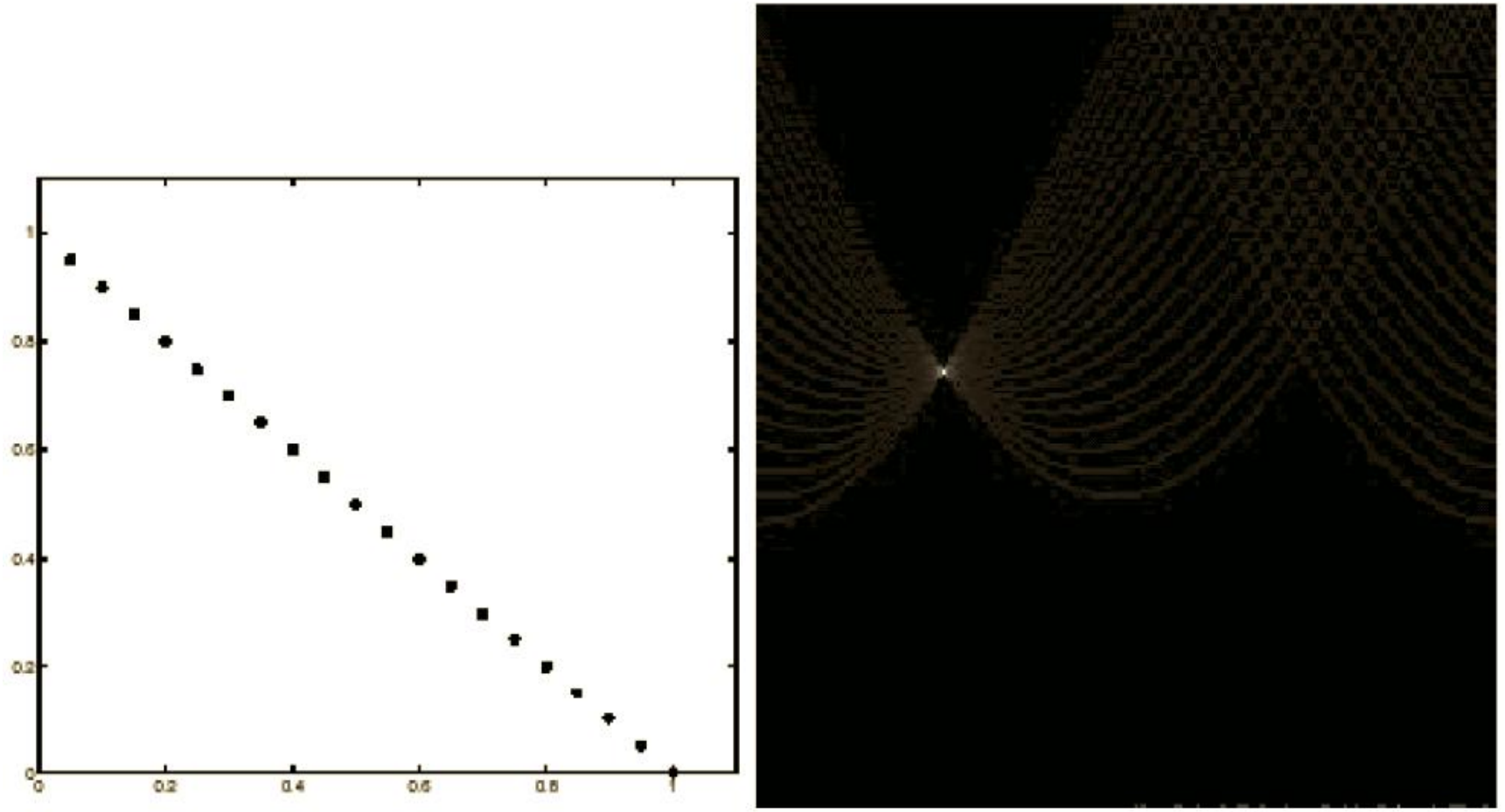


Hough Transform



© M. Chli, P. Furgale, M. Hutter, M. Rufli, D. Scaramuzza, R. Siegwart, Autonomous Mobile Robots, ETH, 2016

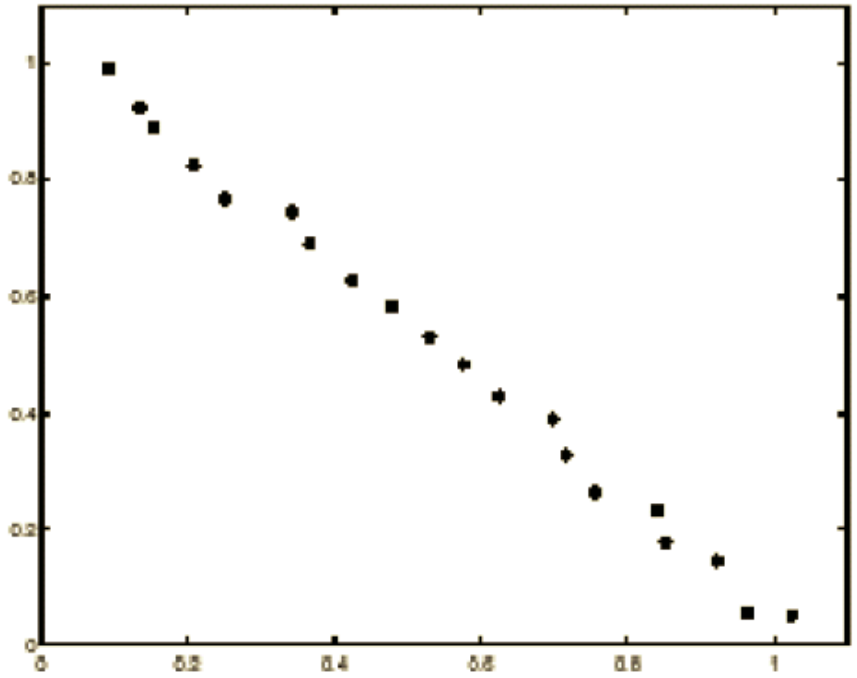
Line extraction: 3. Hough Transform



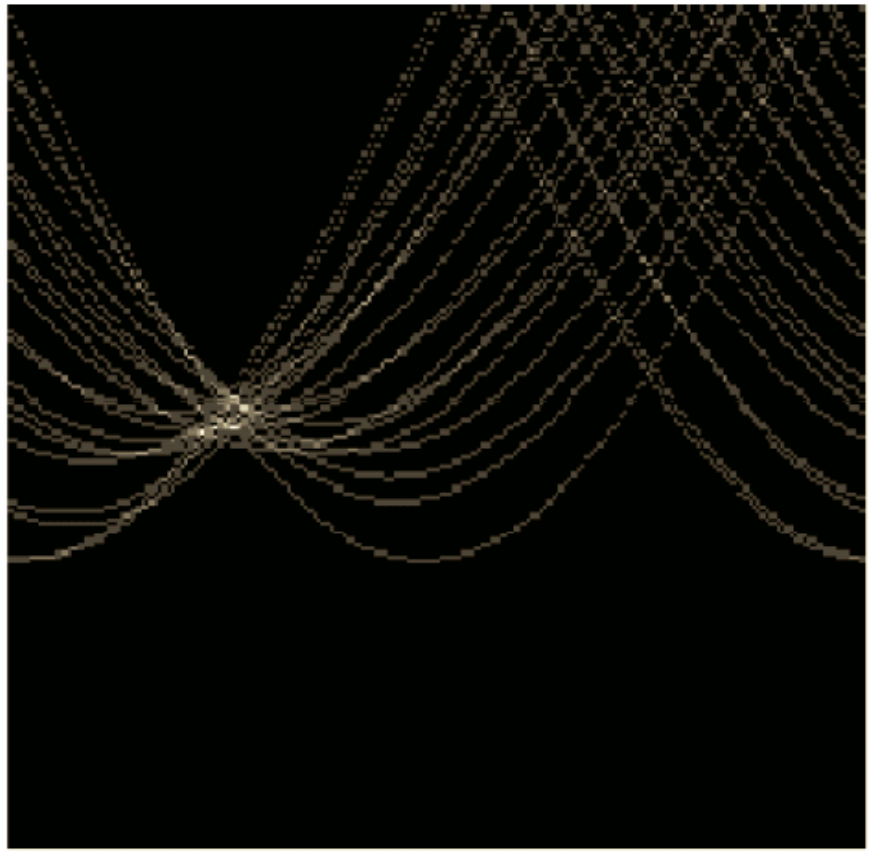
© M. Chli, P. Furgale, M. Hutter, M. Rufli, D. Scaramuzza, R. Siegwart, Autonomous Mobile Robots, ETH, 2016

Line extraction: 3. Hough Transform

Effect of Noise:



features



votes

- Peak gets fuzzy and hard to locate

© M. Chli, P. Furgale, M. Hutter, M. Rufli, D. Scaramuzza, R. Siegwart, Autonomous Mobile Robots, ETH, 2016

Line extraction: 3. Hough Transform

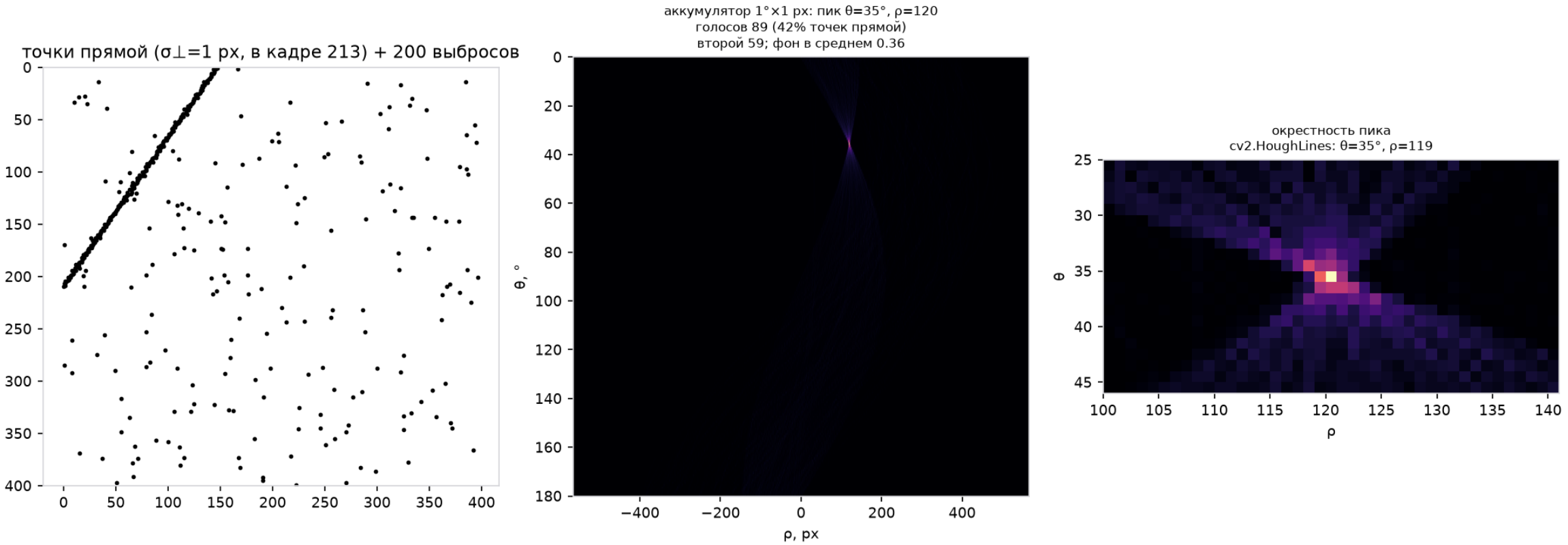
Преобразование Хафа используется для обнаружения прямых линий и других геометрических фигур на изображениях. Однако у него есть несколько недостатков:

1. **Выбор размера сетки**: В преобразовании Хафа используется дискретная сетка для определения параметров линии. Правильный выбор размера ячеек этой сетки является сложной задачей и может сильно повлиять на точность результатов.
2. **Неучет шума и неопределенности**: Преобразование Хафа предполагает, что все точки данных являются точными. На практике, данные могут содержать шум и неопределенности, что приводит к ошибкам в определении параметров линии.
3. **Медленная скорость**: Преобразование Хафа может быть вычислительно затратным и медленным, особенно при обработке изображений с большим количеством точек.

Решение проблемы шума: Использование метода наименьших квадратов внутри преобразования Хафа позволяет учитывать неопределенности и шум в данных, что улучшает точность определения параметров линии.

© M. Chli, P. Furgale, M. Hutter, M. Rufli, D. Scaramuzza, R. Siegwart, Autonomous Mobile Robots, ETH, 2016

Аккумулятор: голосование



Визуализация реального процесса голосования в условиях сильного зашумления (213 точек линии и 200 изолированных точек-выбросов): наглядно доказывает устойчивость метода Хафа. Несмотря на обилие визуального мусора на исходном кадре, в пространстве параметров все равно формируется **один четкий, яркий пик**, позволяющий безошибочно восстановить геометрию исходной прямой.

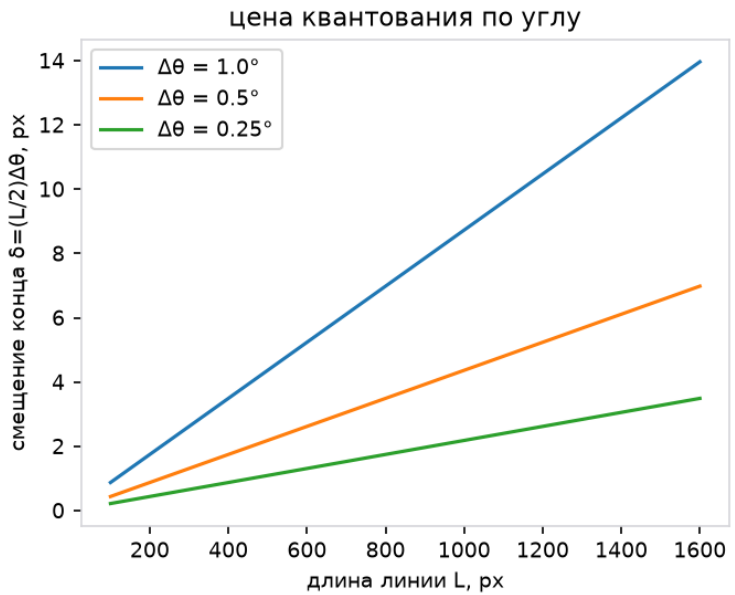
© собственная иллюстрация

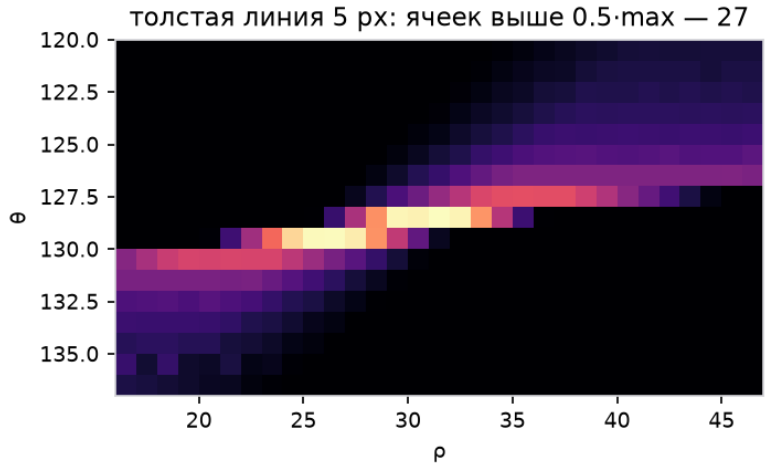
Параметры Хафа в OpenCV

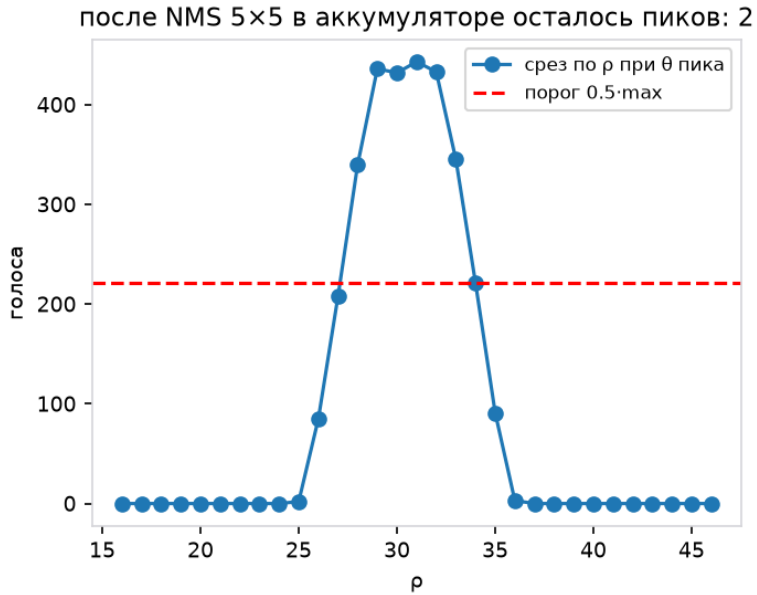
Параметр	Смысл	Типичное	Что если больше
`rho`, `theta`	шаг сетки	1 px, 1° (для длинных линий 0.25–0.5°)	грубее: концы 800 px линии ±7 px при 1°
`threshold`	минимум голосов пика	0.3–0.5 длины линии	меньше — пики-дубли и мусор
`minLineLength` (P)	минимальная длина отрезка	0.2–0.4 диагонали ROI	меньше — обрывки текстуры
`maxLineGap` (P)	максимум разрыва в отрезке	10–25 px	больше — склеит параллельные линии

`HoughLines` даёт (ρ, θ) бесконечных прямых, `HoughLinesP` — отрезки с концами; P быстрее и нужнее роботу

Пики в отрезки: дубли и применения



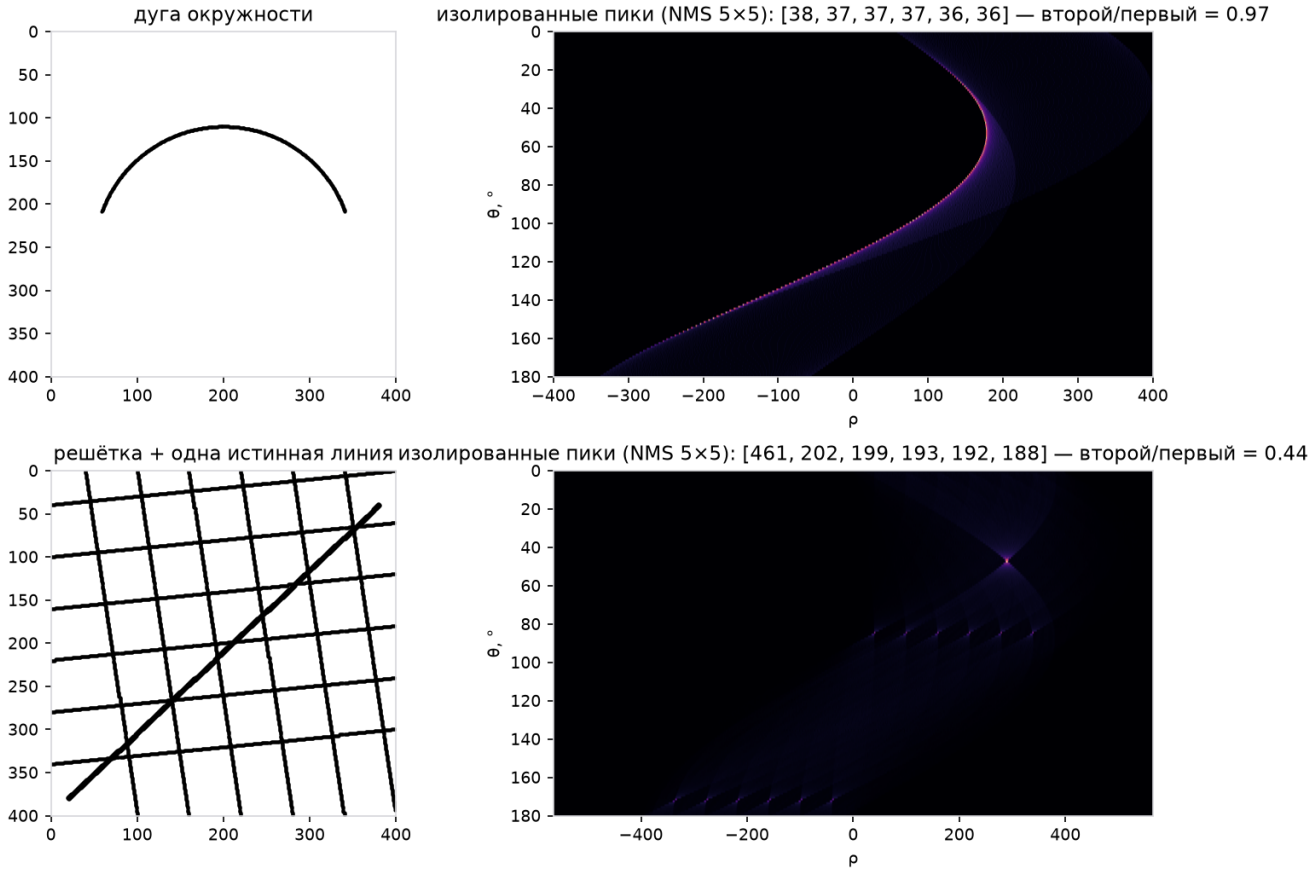




Решение проблемы квантования и размытия пиков. Из-за конечной толщины линий на изображении один реальный объект может порождать целую группу соседних «горячих» ячеек в аккумуляторе. **Подавление немаксимумов (NMS 5x5)** в пространстве аккумулятора позволяет устранить дубликаты векторов и оставить только истинные центральные параметры линий.

© собственная иллюстрация

Вырожденные случаи Хафа



Визуализация, что происходит в пространстве параметров, если на вход подать **дугу окружности** (синусоиды не пересекаются в одной точке, а образуют плавную огибающую) или **решетку из параллельных/пересекающихся линий** (возникает множество ложных локальных максимумов). Это подчеркивает, что классический алгоритм Хафа строго заточен под поиск идеальных прямых.

© собственная иллюстрация

Итог части 2

- Умеем: аккумулятор вручную совпадает с cv2.HoughLines (пик (35°, 120) при 89 голосах); HoughLinesP + кластеризация дают 2 линии разметки за 3.8 мс
- Не умеем: отличить inliers одной прямой от inliers другой, когда прямых много и они пересекаются; аккумулятор растёт с числом моделей
- Если модель одна и данных много с выбросами — дешевле выбирать случайные подмножества: RANSAC

Демо 2.1 — аккумулятор вручную


```python
rng = np.random.default_rng(1); th0, r0 = np.radians(35), 120
s = np.linspace(-180, 180, 300); t = rng.normal(0, 1, 300)
xs = r0*np.cos(th0) - s*np.sin(th0) + t; ys = r0*np.sin(th0) + s*np.cos(th0) + t
xs = np.r_[xs, rng.uniform(0, 400, 200)]; ys = np.r_[ys, rng.uniform(0, 400, 200)]
th = np.deg2rad(np.arange(180)); acc = np.zeros((180, 1133), int)
for i, a in enumerate(th):
    np.add.at(acc[i], np.rint(xs*np.cos(a) + ys*np.sin(a)).astype(int) + 566, 1)
i, j = np.unravel_index(acc.argmax(), acc.shape)
print("пик: θ=%d°, ρ=%d, голосов %d · второй пик %d" % (i, j - 566, acc[i, j], np.sort(acc.ravel())[-2]))
print("cv2.HoughLines по тем же точкам: θ=35°, ρ=119")
```

пик: θ = 35°, ρ = 120, голосов 89 · второй пик 59 · cv2.HoughLines: θ 35°, ρ 119

Вызов	Что делает
`np.add.at(acc[i], rho, 1)`	голосование без цикла по точкам: накопление с повторами индексов
`np.unravel_index(acc.argmax(), acc.shape)`	номер максимума в плоском массиве → (θ, ρ) пика

3

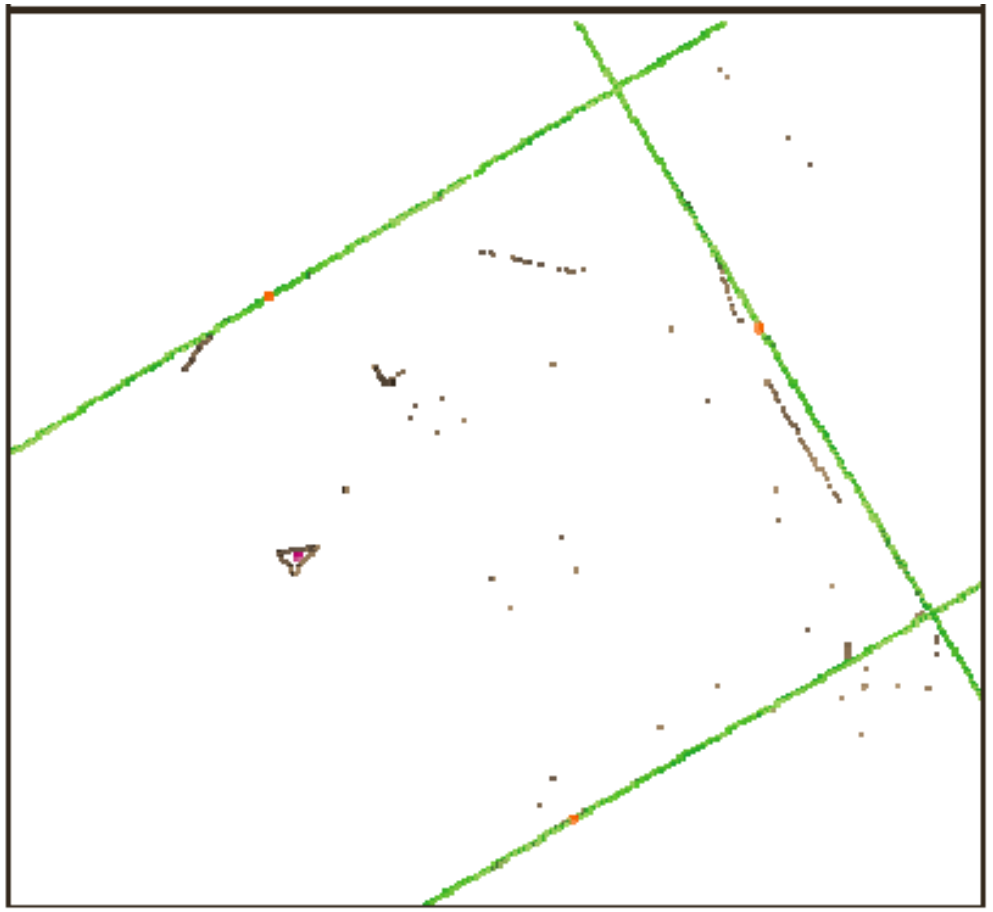
Робастное оценивание: RANSAC

Выбросы и МНК · схема RANSAC · число итераций · Хаф против RANSAC · Радон и Фурье · что живёт в 2026

Line extraction: RANSAC (1)

RANSAC (Random Sampling Consensus) - это метод, который можно использовать для извлечения линий, кривых, поверхностей и даже трехмерных объектов.

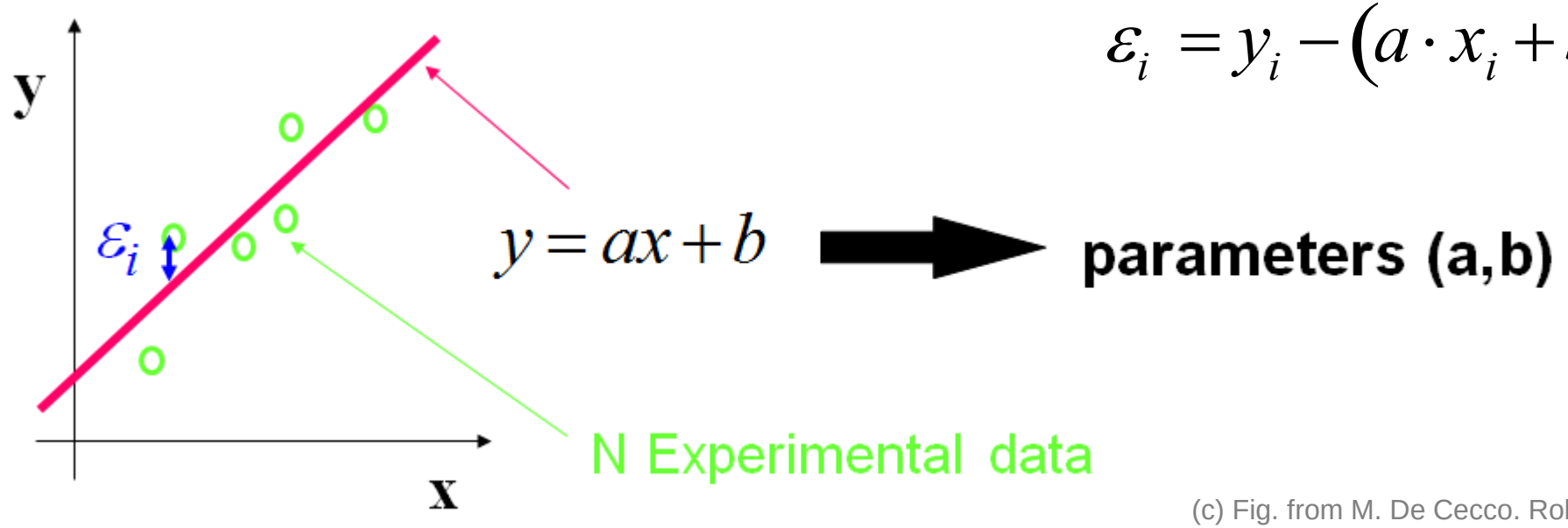
- Внутри помещений лазерные сканеры могут обнаруживать прямые линии, такие как стены, двери и другие вертикальные поверхности. Эти линии могут использоваться как **ориентиры** для навигации роботов.
- **Алгоритм RANSAC** помогает выделить основные линии из данных лазерного сканирования, игнорируя шум и выбросы. Зеленые линии, показанные на изображении, представляют собой наилучшие соответствия, то есть линии, которые наиболее точно отображают положения ориентиров в реальном мире.



(c) Fig. are from S. Riisgaard and M.R. Blas. SLAM for Dummies.

Line extraction: RANSAC (2)

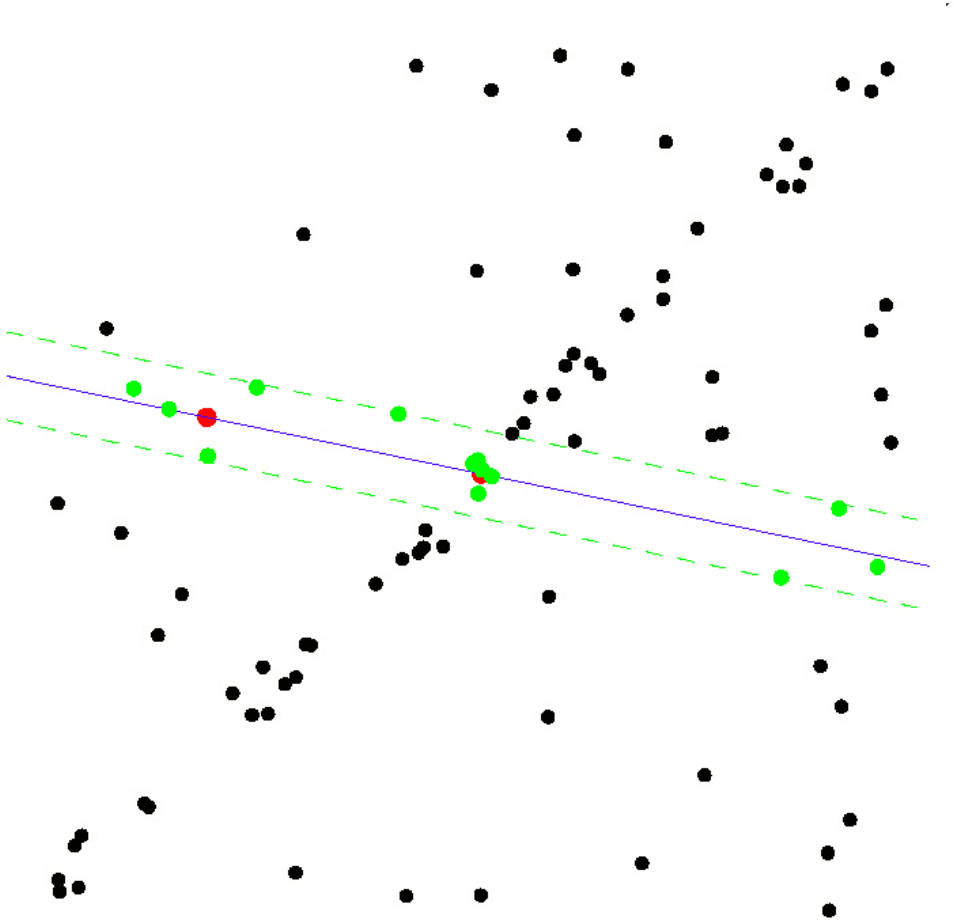
- **Алгоритм RANSAC** (Random Sample Consensus) начинает с выбора случайной подвыборки из всех доступных данных. Затем он применяет метод наименьших квадратов для нахождения линии, которая наилучшим образом описывает эту подвыборку. **Метод наименьших квадратов** минимизирует сумму квадратов отклонений точек от этой линии, чтобы найти наилучшее соответствие.
- **Консенсусный набор** - это набор точек данных, которые соответствуют найденной модели, то есть расположены близко к линии. Алгоритм RANSAC повторяет этот процесс несколько раз, каждую итерацию улучшая модель и добавляя больше точек в консенсусный набор.
- Цель RANSAC - найти модель, которая наилучшим образом описывает данные, несмотря на наличие выбросов (ошибочных данных). Процесс повторяется до тех пор, пока не будет найден наилучший консенсусный набор, что обеспечивает наиболее точное описание данных.



$$\varepsilon_i = y_i - (a \cdot x_i + b)$$

(c) Fig. from M. De Cecco. Robotics and Sensor Fusion

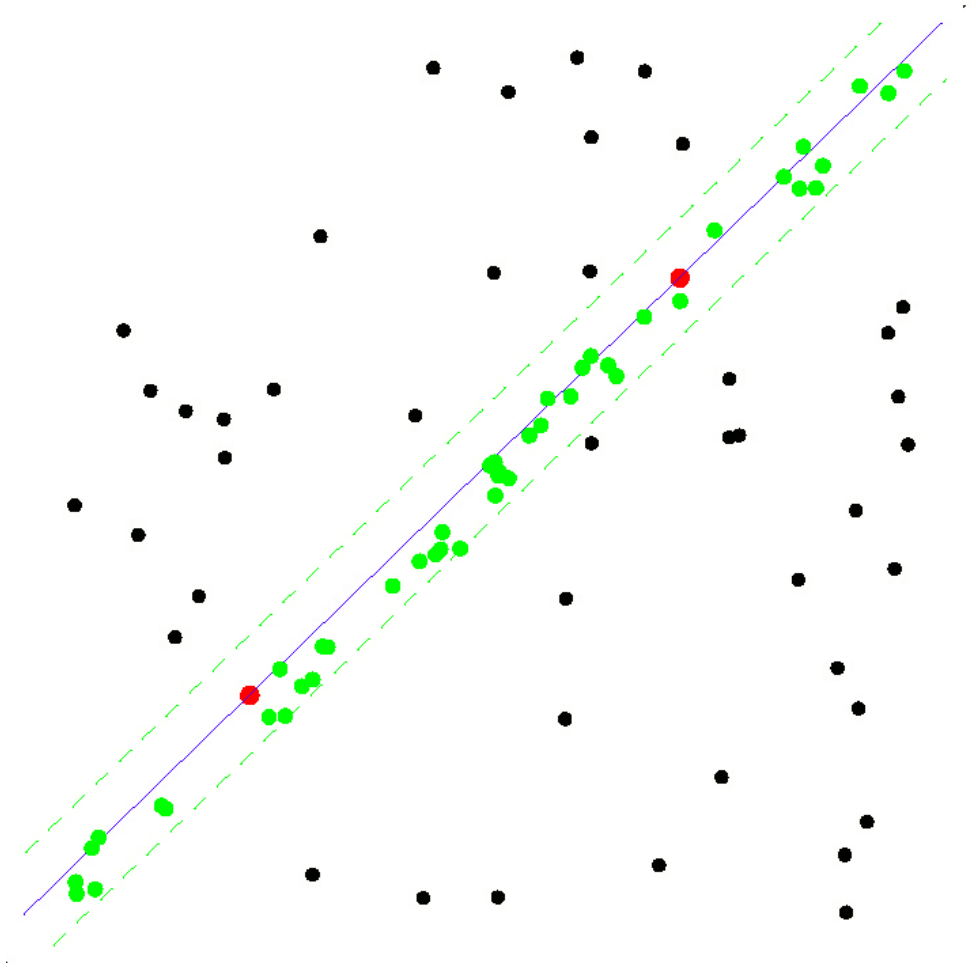
Line extraction: RANSAC (1)



- Select sample of 2 points at random
- Calculate model parameters that fit the data in the sample
- Calculate error function for each data point
- Select data that supports current hypothesis
- **Repeat sampling**

© M. Chli, P. Furgale, M. Hutter, M. Rufli, D. Scaramuzza, R. Siegwart, Autonomous Mobile Robots, ETH, 2016

Line extraction: RANSAC (2)



- Select sample of 2 points at random
- Calculate model parameters that fit the data in the sample
- Calculate error function for each data point
- Select data that supports current hypothesis
- **Repeat sampling**

Set with the maximum number of inliers obtained after k iterations

© M. Chli, P. Furgale, M. Hutter, M. Rufli, D. Scaramuzza, R. Siegwart, Autonomous Mobile Robots, ETH, 2016

Line extraction: RANSAC (3)

How many iterations does RANSAC need?

- Ideally: check all possible combinations of **2** points in a dataset of **N** points.
- Number of all pairwise combinations: **N(N-1)/2**

  ⇨ computationally unfeasible if **N** is too large.

  example: 10'000 points to fit a line through ⇨ need to check all 10'000*9999/2= **50 million** combinations!

- Do we really need to check all combinations or can we stop after some iterations?

  Checking a **subset** of combinations is enough **if** we have a **rough** estimate of the percentage of inliers in our dataset

- This can be done in a probabilistic way

© M. Chli, P. Furgale, M. Hutter, M. Rufli, D. Scaramuzza, R. Siegwart, Autonomous Mobile Robots, ETH, 2016

Line extraction: RANSAC (4)

How many iterations does RANSAC need?

- The number of iterations k is: $$k = \frac{\log(1-p)}{\log(1-w^2)}$$

⇨ knowing the fraction of inliers w, after k RANSAC iterations we will have a probability p of finding a set of points free of outliers

- Example: if we want a probability of success p=99% and we know that w=50% ⇨ k=16 iterations – these are dramatically fewer than the number of all possible combinations!

- Notice: the number of points does not influence the estimated number of iterations, only w does!

- In practice we need only a rough estimate of w.
  More advanced variants of RANSAC estimate the fraction of inliers and adaptively change it on every iteration

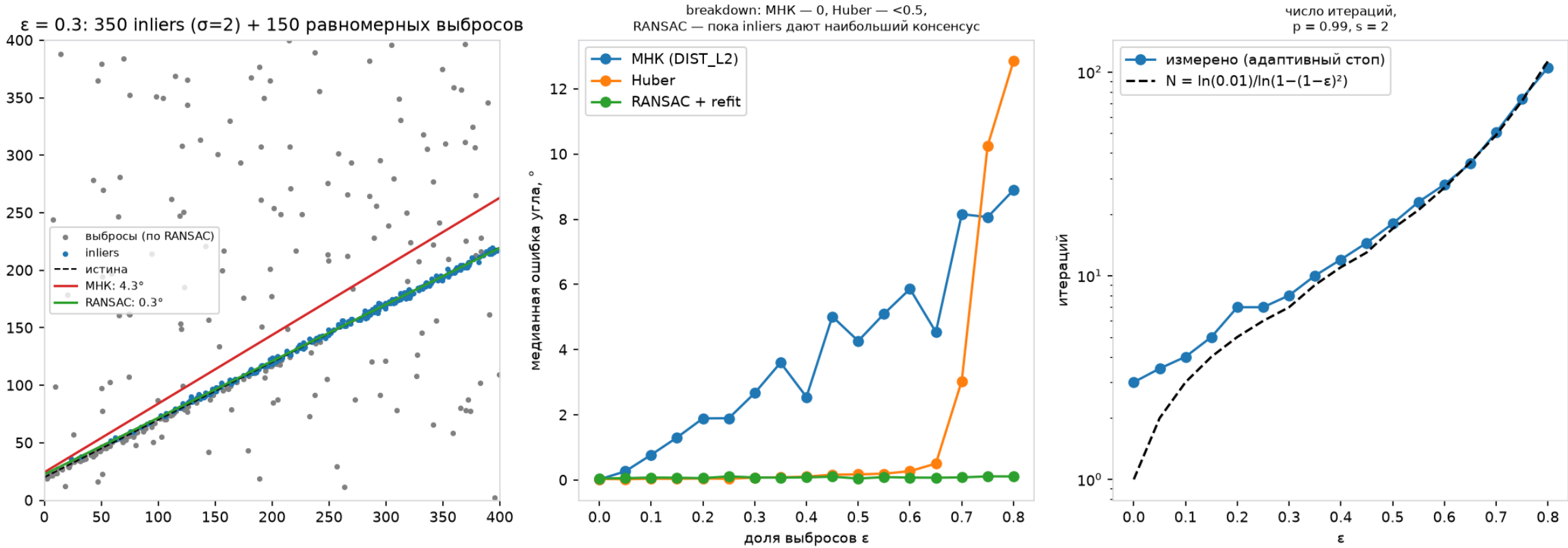
© M. Chli, P. Furgale, M. Hutter, M. Rufli, D. Scaramuzza, R. Siegwart, Autonomous Mobile Robots, ETH, 2016

Line extraction: 3. RANSAC (5)

RANSAC (Random Sample Consensus) - это алгоритм, который используется для оценки параметров модели из набора данных, содержащих выбросы.

- **Основное преимущество RANSAC** заключается в универсальности: он может работать с различными типами признаков, если у нас есть модель этих признаков.
- **Выбросы** - это данные, которые значительно отклоняются от остальных значений и могут исказить результаты анализа. RANSAC хорошо справляется с большим количеством выбросов (более 50%), что делает его полезным в обработке реальных данных, которые часто содержат шум.
- **Многократное извлечение линий**: Если требуется извлечь несколько линий, алгоритм RANSAC следует запускать несколько раз, каждый раз удаляя уже найденные линии, чтобы найти новые.
- **Ограничения**: Недостатком RANSAC является то, что при достижении максимального количества итераций алгоритм может не найти оптимальное решение. Кроме того, найденное решение может не быть наилучшим образом подходящим для данных.

МНК против выбросов: breakdown 0 %



Демонстрация критической уязвимости классического **Метода Наименьших Квадратов (МНК)**. Его точка слома (breakdown point) равна 0% — достаточно всего одного сильного выброса, чтобы полностью исказить результаты прямой регрессии. Графики показывают, как стандартный МНК (LS) уходит вслед за шумом, в то время как робастные функции потерь (например, Хубера) удерживают верное направление.

© собственная иллюстрация

Итерации: $N = \ln(1-p)/\ln(1-\varepsilon^s)$

Доля выбросов ε	ε^s (s = 2)	N при p = 0.99	N при p = 0.999
0.1	0.81	3	4
0.3	0.49	7	10
0.5	0.25	17	25
0.7	0.09	49	70
0.9	0.01	459	688

Адаптивный стоп: после каждой удачной выборки пересчитать оценку доли inliers и уменьшить N; для круга s = 3 — при ε = 0.6 нужно 70 итераций

Практическая таблица зависимости числа итераций *N* от доли выбросов ε и вероятности успеха *p*. Инженерный прием — адаптивный стоп: изначально мы не знаем точную долю шума, но после каждой случайной выборки, которая дала хорошую модель, мы можем пересчитывать оценку количества инлайеров и динамически уменьшать требуемое число итераций *N* прямо во время работы алгоритма.

Хаф против RANSAC · куда дальше

	Хаф	RANSAC
Кто голосует	все точки во все совместимые ячейки	случайные минимальные выборки
Результат	все модели сразу (все пики)	одна модель за проход (повтор с удалением inliers — для нескольких)
Точность	ограничена сеткой; уточнение отдельно	не ограничена; refit по inliers
Память	O(∏ N_θk), экспоненциально по s	O(n)
Робастность	порог голосов	t и N; breakdown до 1 − w_min
Когда	много прямых/окружностей одновременно, s ≤ 3	одна модель, s ≥ 2, большая доля выбросов

Правило: несколько линий — Хаф (+ уточнение по точкам пика); одна модель с выбросами — RANSAC; гибрид — Хаф для инициализации кластеров, RANSAC внутри кластера

Дальше по курсу: L10 (`findHomography`), L12 (fundamental matrix), L14 (трекинг); в OpenCV 4.x RANSAC собран во фреймворк USAC (`USAC_MAGSAC`, `USAC_ACCURATE`: LO-RANSAC, PROSAC, SPRT, MAGSAC++ без жёсткого t)

Сравнение детекторов линий

- Детерминированные алгоритмы: Split-and-Merge, Incremental, Regression, Hough transform
- Недетерминированные алгоритмы: RANSAC, EM (Expectation Maximization)
- RANSAC и EM недетерминированы, потому что их результаты могут отличаться при каждом запуске. Это потому, что эти два алгоритма генерируют случайные гипотезы
- Split-and-Merge, Incremental и Regression: самые быстрые – они применяются при сканировании лидаром
- RANSAC, Hough transform и EM produce имеют более высокую точность → они более устойчивы к выбросам (outliers)

N: количество точек во входном сканировании
S: количество извлеченных сегментов линии
N_f: размер скользящего окна для линейной регрессии
k: Количество циклов RANSAC
N_C, N_R : Количество столбцов и строк для массива аккумулятора Хафа (Hough)
N_1, N_2 : количество испытаний и итераций сходимости для алгоритма EM

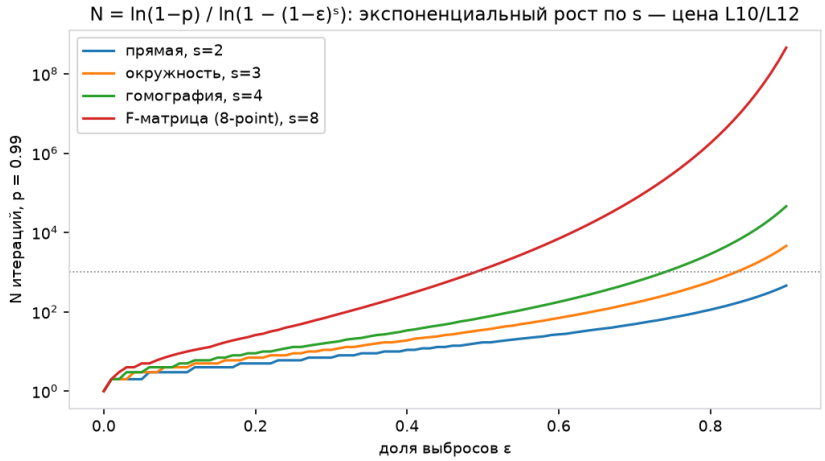
	Complexity	Speed (Hz)	False positives	Precision
Split-and-Merge	N logN	1500	10%	+++
Incremental	$S\,N^2$	600	6%	+++
Line-Regression	$N\,N_f$	400	10%	+++
RANSAC	$S\,N\,k$	30	30%	++++
Hough-Transform	$S\,N\,N_C + S\,N_R\,N_C$	10	30%	++++
Expectation Maximization	$S\,N_1\,N_2\,N$	1	50%	++++

[Nguyen et al. IROS 2005]

© M. Chli, P. Furgale, M. Hutter, M. Rufli, D. Scaramuzza, R. Siegwart, Autonomous Mobile Robots, ETH, 2016

Демо 3.1 — breakdown и итерации


```python
def ransac_line(p, t, p_ok=0.99):
    best, N, k = np.zeros(len(p), bool), 1000, 0
    while k < N:
        a, b = p[rng.choice(len(p), 2, replace=False)]; k += 1
        inl = line_dist(p, a, b) < t
        if inl.sum() > best.sum():
            best = inl; N = int(np.log(0.01) / np.log(1 - inl.mean() ** 2)) + 1
    return np.polyfit(p[best, 0], p[best, 1], 1), best
for eps in (0.0, 0.3, 0.5, 0.7):
    p = make_line(500, eps, 2.0)     # y = 0.5x + 20 + шум, выбросы равномерные
    e_ls = ang_err(*np.polyfit(p[:, 0], p[:, 1], 1)); (vr, inl) = ransac_line(p, 4.0)
    print("ε %.1f: МНК %4.1f° · RANSAC %.2f° · inliers %.0f %%"
                    % (eps, e_ls, ang_err(*vr), 100*inl.mean()))
```



Вызов	Что делает
`line_dist(p, a, b)`	расстояние от всех точек до прямой через двухточечную модель: консенсус при d < t
`N = ln(0.01)/ln(1−w²)+1`	адаптивный стоп: w — доля inliers лучшей модели, p = 0.99, s = 2

© собственная иллюстрация

Демо 3.2 — реальная полоса: L2, Huber, RANSAC

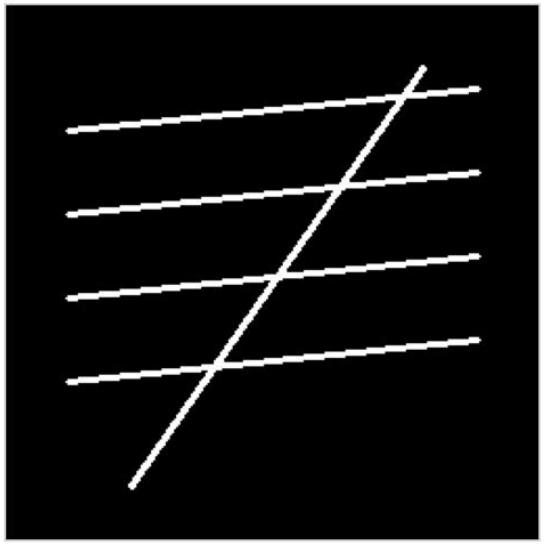

```python
pts = edge_points(road, roi_left)                    # 1840 px края левой полосы + мусор
ls  = cv2.fitLine(pts, cv2.DIST_L2, 0, 0.01, 0.01)
hub = cv2.fitLine(pts, cv2.DIST_HUBER, 0, 0.01, 0.01)
ran, inl = ransac_line(pts.reshape(-1, 2), 2.5)
ref = cv2.fitLine(pts[inl], cv2.DIST_L2, 0, 0.01, 0.01)
print("выбросов %.0f %% · ошибка угла: L2 %.1f° · Huber %.1f° · RANSAC+refit %.1f°"
      % (100 * (1 - inl.mean()), ang_err(ls), ang_err(hub), ang_err(ref)))
```

выбросов 34 % · ошибка угла: L2 4.3° · Huber 0.0° · RANSAC+refit 0.3° (inliers 66 %)

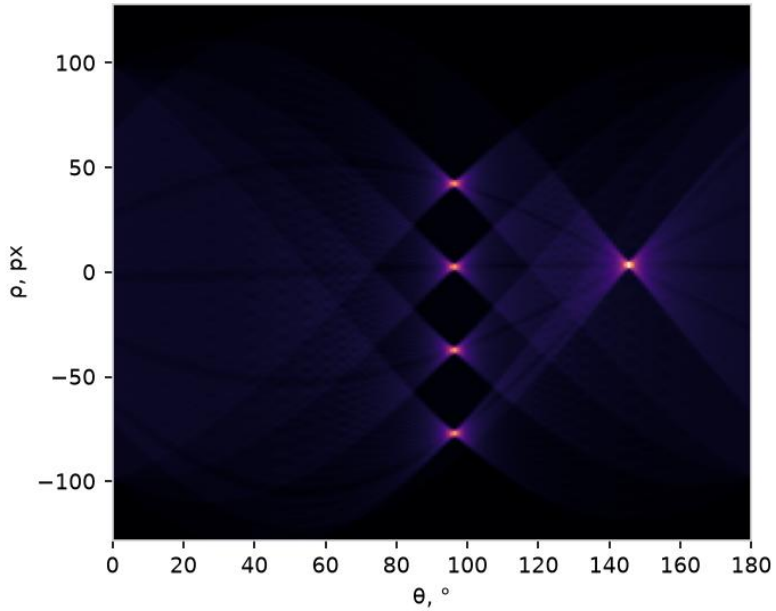
Вызов	Что делает
`cv2.fitLine(pts, cv2.DIST_HUBER, …)`	M-оценка: влияние больших невязок ограничено, решается IRLS
`cv2.fitLine(pts[inl], …)`	refit по консенсусу: RANSAC отобрал точки, МНК их дооценил

Радон, Хаф и slice-теорема (обзор)

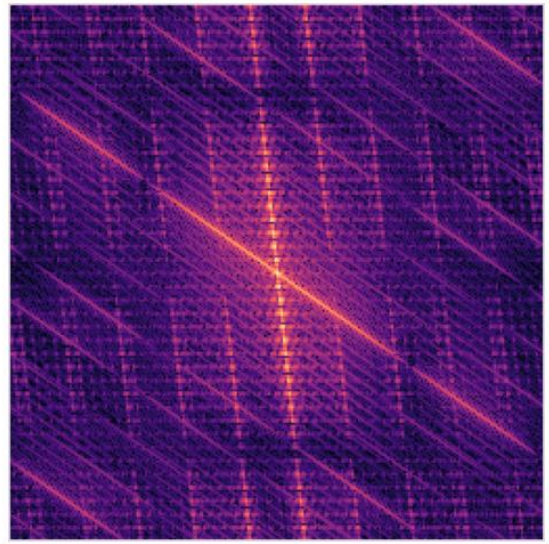
кадр: 4 параллельные + 1 поперечная прямая



синограмма (Радон): прямая → яркая точка



log|FFT|: прямая → луч, перпендикулярный ей (slice-теорема)



Связь компьютерного зрения с обработкой сигналов и томографией. Показывается сходство преобразования Хафа с **преобразованием Радона** (интегрирование яркости вдоль прямых линий), которое преобразует параллельные прямые в яркие точки на синограмме. Через **slice-теорему (теорему о центральном сечении)** демонстрируется связь этих пространств с **двумерным преобразованием Фурье (FFT)**, где прямая линия переходит в перпендикулярный ей луч в частотной области.

© собственная иллюстрация

Что пришло на смену к 2026 году

- HED и deep edge detection (2015–): контуры как задача сегментации, BSDS ODS-F 0.80–0.83 против 0.61 у Canny; цена — GPU и 30 мс
- EDLines / LSD: детекторы отрезков без аккумулятора, точнее на коротких линиях; LSD возвращён в OpenCV contrib (ximgproc)
- USAC (Raguram 2013), MAGSAC++ (2020): в 2–5 раз быстрее классики на больших ε, без ручного t; всё внутри `cv2.findHomography`
- Где классика жива: Canny — базовый препроцессинг и обучение без GPU; Хаф — линии разметки и сетки калибровки; RANSAC — каждая оценка модели в L10–L14

Итог части 3

- Умеем: отличать выброс от шума; считать итерации RANSAC (ε 0.5 → 17, ε 0.9 → 459 при s = 2); выбирать L2 / Huber / RANSAC по доле выбросов (4.3° / 0.0° / 0.3°)
- Блок III начат: края (L7) → точки интереса (L8) → дескрипторы (L9); робастность сквозная: L10, L12
- Открытый вопрос: край и прямая найдены по яркости, а угол — это уже две прямые; можно ли найти угол, не строя прямых? L8 — Харрис и Ши-Томаси

Разбор: что не так с этим кодом

Фрагмент «найти линию края»

- `e = cv2.Canny(noisy, 40, 110)` — noisy: σ шума 10
- `ln = cv2.HoughLines(gray, 1, np.pi/180, 120)`
- `k, b = np.polyfit(xs, ys, 1)` — край забора на кадре
- `m = cv2.fitLine(pts, cv2.DIST_L2, 0, 0.01, 0.01)` — 40 % бликов

Четыре ошибки

- Canny без сглаживания на σ 10: 18 % пикселей кадра становятся «краем», SNR 5.0; нужен Гаусс σ ≈ 1 до
- HoughLines по серому изображению: голосуют все ненулевые пиксели — тысячи ложных прямых; вход — бинарные края
- polyfit для вертикальной прямой: k → ∞, численно взрыв; параметризация (ρ, θ) или fitLine
- DIST_L2 при 40 % выбросов: ошибка 4.3°; Huber или RANSAC (демо 3.2)

Порядки величин (1920 × 1279)

Операция	Время	Замечание
Собель (обе оси) + magnitude	2.5–3.5 мс	uint8 → CV_64F; Scharr столько же
`cv2.Canny`	4–6 мс	включает Гаусс? нет: сглаживание отдельно, +1 мс
closing 3×3 + findContours + approxPolyDP	1.5 + 2–4 + 0.1 мс	контуров < 100
`HoughLinesP` после Canny с ROI	1–4 мс	без ROI до 15 мс и 61 отрезок
RANSAC прямая, 2k точек, N ≤ 500	0.5–2 мс	numpy-версия демо 3.1
`cv2.fitLine`	0.05 мс	любой DIST_*
Canny вручную (numpy, демо 1.2)	20–30 мс	цена понятности

Что унести с собой

KEY TAKEAWAYS

- Край — производная после сглаживания: Собель = [1 2 1]⊗[1 0 −1], SNR пика 3.3 → 5.0 → 7.8; вторая производная даёт замкнутые нуль-переходы, но шумит квадратично
- Canny = градиент + NMS + гистерезис: пороги из Otsu по \|∇I\| (фикс. при смене света 0.81 → 0.0 F1, динам. 1.0/0.91); вручную совпадает с cv2.Canny на 1.0
- Прямая — голосование в (ρ, θ): выбросы не создают пика; квант θ 1° даёт ±7 px на концах 800 px; HoughLinesP + кластеризация: 17 отрезков → 2 линии за 3.8 мс
- RANSAC: $N = \ln(1-p)/\ln(1-\varepsilon^s)$, ε 0.5 → 17 итераций, ε 0.9 → 459; МНК ломается на ε 0.3 (0.8°), RANSAC+refit держит 0.08° при 34 % выбросов
- Мосты: ориентации градиента → L8/L9, RANSAC → L10/L12, контуры и прямые → ДЗ 2 «Сканер документов»

Вопросы для самопроверки

САМОПРОВЕРКА

- Почему Собель сглаживает поперёк производной? Как выглядит его спектр (L6, слайд 16) и что изменится без сглаживания?
- Кадр стал вдвое темнее и вдвое малоконтрастнее. Какие параметры конвейера Canny менять и почему σ остаётся прежней?
- Отрезок 500 px, сетка Хафа 1 px × 1°. Оцените разброс концов отрезка; как уменьшить его, не увеличивая память аккумулятора вчетверо?
- Модель — окружность (s = 3), выбросов 60 %. Сколько итераций RANSAC нужно при p = 0.99? Что даст адаптивная оценка доли inliers?

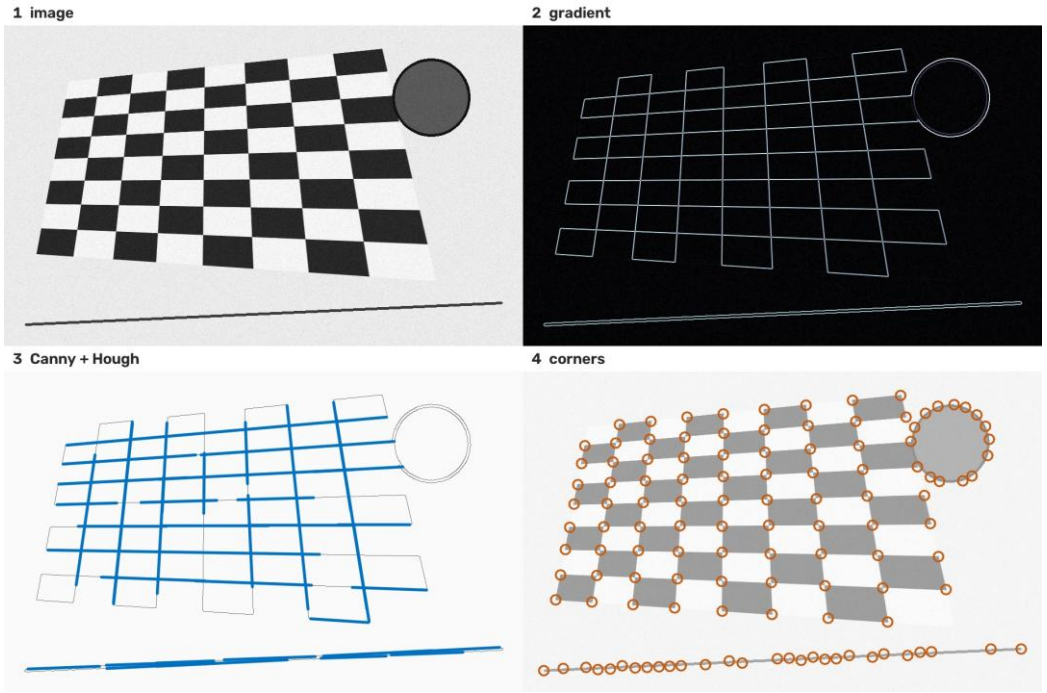
Разбор — в начале L8, на слайде «Опоры»

References

REFERENCES

- Szeliski R. Computer Vision: Algorithms and Applications, 2nd ed., 2022 — §7.2 (края), §7.4 (прямые и робастное оценивание); к L8 — §7.1 (точки интереса)
- Canny J. A Computational Approach to Edge Detection, IEEE TPAMI, 1986 · Duda R., Hart P. Use of the Hough Transform to Detect Lines and Patterns, CACM, 1972
- Fischler M., Bolles R. Random Sample Consensus, CACM, 1981 · Raguram R. et al. USAC: A Universal Framework for Random Sample Consensus, IEEE TPAMI, 2013
- Гонсалес Р., Вудс Р. Цифровая обработка изображений, 3-е изд., 2012 — гл. 9 (морфология), гл. 10 (сегментация по краям)
- OpenCV docs 4.x: `Canny`, `Sobel`/`Scharr`, `findContours`/`approxPolyDP`, `HoughLines`/`HoughLinesP`, `fitLine` (DIST_*), USAC-флаги RANSAC

Вопросы?



Dr. Ilya Afanasyev · i.afanasyev@innopolis.ru